UNIVERSIDADE MUNICIPAL DE SÃO CAETANO DO SUL – USCS
Bacharelado em Ciência da Computação

MANUAL DE REFERÊNCIA

Redes de Computadores

Material explicativo dos tópicos da Matriz de Referência,
alinhado às DCN de Computação (Resolução CNE/CES nº 5/2016)

SQUAD 1

Guilherme Ayres
Henrique Martins
João Pedro Lourenço
Leonardo Moraes
Stefano Augusto

São Caetano do Sul – SP

Setembro de 2026

Sumário

Apresentação e Alinhamento com as DCN **3**

1 Fundamentos de Redes de Computadores e Modelo de Referência OSI **4**

1.1 Componentes de uma rede 4

1.2 Classificação das redes por abrangência geográfica 5

1.3 Comutação de circuitos e comutação de pacotes 5

1.4 Métricas de desempenho: atraso, perda e vazão 6

1.5 Protocolos e arquitetura em camadas 7

1.6 O Modelo de Referência OSI 8

2 Modelo TCP/IP e Encapsulamento de Dados **10**

2.1 As camadas do modelo TCP/IP 10

2.2 O modelo “ampulheta” 11

2.3 Encapsulamento de dados 11

2.4 Eficiência e sobrecarga (overhead) 12

2.5 Acompanhando um pacote de ponta a ponta 13

3 Meios de Transmissão, Topologias e Equipamentos de Interconexão **13**

3.1 Conceitos básicos de transmissão 13

3.2 Meios guiados 14

3.3 Meios não guiados (sem fio) 15

3.4 Topologias de rede 15

3.5 Equipamentos de interconexão 17

4 Endereçamento IP, Máscaras de Sub-rede e IPv6 **19**

4.1 O datagrama IPv4 19

4.2 Estrutura do endereço IPv4 19

4.3 Máscara de sub-rede e CIDR 20

4.4 Divisão em sub-redes (subnetting) 21

4.5 Endereços especiais, privados e NAT 22

4.6 IPv6 23

5 Protocolos de Roteamento e Camada de Enlace **25**

5.1 Encaminhamento e roteamento 25

5.2 Algoritmos de roteamento 26

5.3 Roteamento na Internet: sistemas autônomos 28

5.4 Camada de enlace 28

6 Camada de Transporte: Protocolos TCP e UDP **31**

6.1 Serviços da camada de transporte 31

6.2 Portas e sockets 31

6.3 UDP: User Datagram Protocol . . . 32
6.4 Princípios de transferência confiável . . . 32
6.5 TCP: Transmission Control Protocol . . . 33
6.6 TCP × UDP: quadro-resumo . . . 37

7 Camada de Aplicação e Principais Protocolos (HTTP, DNS, SMTP, FTP) 37
7.1 Arquiteturas de aplicação . . . 37
7.2 HTTP: a Web . . . 37
7.3 DNS: o catálogo de nomes da Internet . . . 39
7.4 Correio eletrônico: SMTP, POP3 e IMAP . . . 41
7.5 FTP e transferência de arquivos . . . 42
7.6 Outros protocolos de aplicação importantes . . . 43

8 Segurança de Redes: Firewalls, VPNs e Criptografia 43
8.1 Propriedades de uma comunicação segura . . . 43
8.2 Principais ameaças . . . 43
8.3 Fundamentos de criptografia . . . 44
8.4 Firewalls . . . 47
8.5 VPNs: redes privadas virtuais . . . 48
8.6 Defesa em profundidade e Zero Trust . . . 49

9 Redes Sem Fio, Mobilidade e Tendências (Wi-Fi, 4G/5G, SDN, Nuvem e IoT) 49
9.1 Características dos enlaces sem fio . . . 49
9.2 Redes locais sem fio: IEEE 802.11 (Wi-Fi) . . . 50
9.3 Redes celulares: do 1G ao 5G . . . 51
9.4 Mobilidade . . . 53
9.5 Redes Definidas por Software (SDN) . . . 53
9.6 Computação em nuvem e redes . . . 54
9.7 Internet das Coisas (IoT) . . . 55

Referências 57

Apresentação

Este manual reúne, em um único documento, o material explicativo dos nove temas que compõem a Matriz de Referência de Redes de Computadores trabalhada pelo nosso grupo. A ideia não é substituir os livros-texto, mas organizar o conteúdo de forma progressiva, partindo dos conceitos mais gerais (o que é uma rede e como ela é organizada em camadas) até os temas mais atuais (5G, redes definidas por software, computação em nuvem e Internet das Coisas).

Diferente dos slides, que servem de apoio visual durante a apresentação, aqui cada tópico é desenvolvido com mais profundidade: há definições formais, fórmulas, tabelas comparativas, esquemas, exemplos resolvidos passo a passo, analogias do dia a dia e questões comentadas no estilo do ENADE. Ao final de cada figura e tabela indicamos o referencial teórico de onde o conteúdo foi adaptado, e a lista completa das obras consultadas está na seção de Referências.

Como o material está organizado

A sequência dos capítulos segue a lógica da pilha de protocolos, aproximando-se da abordagem de Tanenbaum [2] nos primeiros capítulos (de baixo para cima, começando pelos meios físicos) e da abordagem de Kurose e Ross [1] nos capítulos de transporte e aplicação. Os nove temas estão distribuídos assim:

Tabela 1: Organização dos capítulos e camadas envolvidas

Cap.	Tema da Matriz de Referência	Camada(s) predominante(s)
1	Fundamentos de redes de computadores e modelo de referência OSI	Visão geral (todas)
2	Modelo TCP/IP e encapsulamento de dados	Visão geral (todas)
3	Meios de transmissão, topologias e equipamentos de interconexão	Física e Enlace
4	Endereçamento IP, máscaras de sub-rede e IPv6	Rede
5	Protocolos de roteamento e camada de enlace	Rede e Enlace
6	Camada de transporte: protocolos TCP e UDP	Transporte
7	Camada de aplicação e principais protocolos (HTTP, DNS, SMTP, FTP)	Aplicação
8	Segurança de redes: firewalls, VPNs e criptografia	Transversal
9	Redes sem fio, mobilidade e tendências	Transversal

Fonte: elaborado pelo SQUAD 1.

Alinhamento com as Diretrizes Curriculares Nacionais

A Resolução CNE/CES nº 5, de 16 de novembro de 2016 [9], institui as Diretrizes Curriculares Nacionais (DCN) para os cursos da área de Computação. No Art. 4º, § 1º, o documento espera que o egresso de Ciência da Computação “conheça a estrutura dos sistemas de computação e os processos envolvidos na sua construção e análise” e que “domine os fundamentos teóricos da área”. Redes de computadores é justamente um dos lugares em que teoria (teoria da informação, grafos, filas, criptografia) e prática (configuração, dimensionamento e segurança de infraestrutura) se encontram de forma mais evidente.

A Tabela 2 relaciona cada capítulo com as competências e habilidades listadas no Art. 5º da resolução. Os incisos sem indicação de parágrafo referem-se às competências comuns (Art. 5º, *caput*); os marcados com “§1º” são específicos do bacharelado em Ciência da Computação.

> **Dica de estudo**
>
> Leia cada capítulo tentando responder a três perguntas: **(1)** qual problema essa camada ou tecnologia resolve? **(2)** que informação ela acrescenta (cabeçalho, endereço, número de sequência etc.)? **(3)** o que acontece quando ela falha? Quem consegue responder essas três

Tabela 2: Mapeamento entre os temas e as competências da Resolução CNE/CES nº 5/2016

Cap.	Competência da DCN (Art. 5º)	Como o tema desenvolve a competência
1	V – compreender e explicar as dimensões quantitativas de um problema; §1º, X – aplicar abstração e complexidade	Cálculo de atrasos, vazão e produto banda-atraso; o modelo em camadas como exemplo clássico de abstração.
2	§1º, I – compreender conceitos, princípios e teorias da Computação	Entender o encapsulamento e por que a Internet adotou a pilha TCP/IP.
3	IV – decidir com base no funcionamento do hardware e na infraestrutura, considerando impactos	Escolha de meios físicos e equipamentos; limites de Nyquist e Shannon.
4	I – identificar problemas com solução algorítmica; §1º, IV – analisar requisitos e planejar soluções	Planejamento de sub-redes (VLSM), operações binárias e transição para IPv6.
5	I e II – solução algorítmica e limites da computação; §1º, X – complexidade	Algoritmos de Dijkstra e Bellman-Ford, problema da contagem ao infinito, detecção de erros por CRC.
6	§1º, X – compartilhamento de recursos e concorrência	Controle de fluxo e de congestionamento como divisão justa de um recurso compartilhado.
7	§1º, X – localidade de referência (*caching*); §1º, XII – recuperação da informação	Cache HTTP e cache de DNS; organização hierárquica e distribuída de nomes.
8	§1º, III – identificar e gerenciar riscos, incluindo dependabilidade e segurança; I (Art. 4º) – questões éticas e legais	Criptografia, firewalls, VPNs e defesa em profundidade; LGPD e responsabilidade profissional.
9	IX – adequar-se rapidamente às mudanças tecnológicas; §1º, II – pensamento computacional em domínios diversos	5G, SDN, nuvem e IoT como tendências que mudam a forma de projetar sistemas.

Fonte: elaborado pelo SQUAD 1 a partir da Resolução CNE/CES nº 5/2016 [9].

> perguntas para cada tema geralmente vai bem nas questões de redes do ENADE, porque elas quase sempre cobram o *porquê* das coisas, e não só a definição decorada.

1 Fundamentos de Redes de Computadores e Modelo de Referência OSI

Uma **rede de computadores** é um conjunto de dispositivos autônomos interconectados por enlaces de comunicação e que trocam informações seguindo regras comuns, chamadas **protocolos** [2]. Essa definição tem três palavras-chave. *Autônomos* significa que cada equipamento tem capacidade de processamento própria (não é um terminal "burro" dependente de um computador central). *Interconectados* significa que existe algum meio físico ou sem fio que liga esses equipamentos. E *protocolos* são o acordo que permite que máquinas de fabricantes, sistemas operacionais e linguagens diferentes se entendam.

A Internet é o maior exemplo de rede que existe: uma "rede de redes" formada por bilhões de dispositivos, chamados de **sistemas finais** ou **hospedeiros** (*hosts*), ligados por **enlaces** e por **comutadores de pacotes** (roteadores e switches) [1].

1.1 Componentes de uma rede

Kurose e Ross [1] propõem olhar a rede sob duas perspectivas complementares:

1. **Visão dos componentes ("porcas e parafusos")**: a rede é formada por hardware e software

concretos – computadores, celulares, servidores, cabos, fibras, antenas, roteadores, switches e os programas que implementam os protocolos.

2. **Visão de serviço**: a rede é uma infraestrutura que oferece serviços para as aplicações distribuídas (e-mail, web, streaming, jogos online). A aplicação não precisa saber por onde os dados passam; ela apenas usa uma interface (os *sockets*) para pedir que os dados sejam entregues.

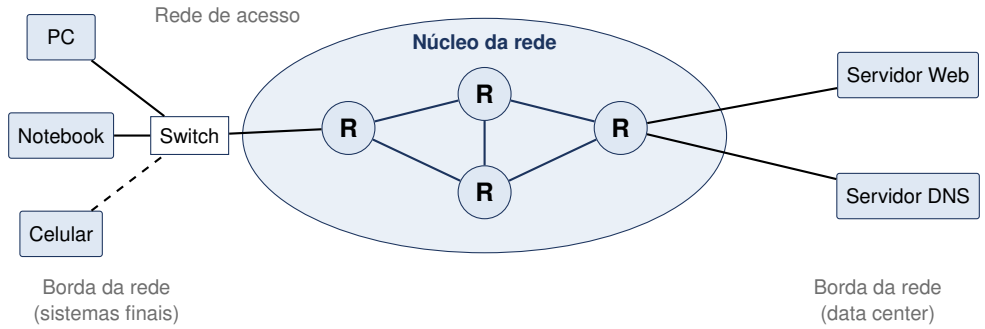


Figura 1: Visão geral de uma rede: borda (sistemas finais), rede de acesso e núcleo (roteadores)

Fonte: adaptado de Kurose e Ross [1].

A Figura 1 mostra a divisão clássica em três regiões: a **borda da rede**, onde ficam os sistemas finais que executam as aplicações; as **redes de acesso**, que conectam os sistemas finais ao primeiro roteador (Wi-Fi doméstico, rede cabeada da empresa, 4G/5G, fibra FTTH); e o **núcleo da rede**, uma malha de roteadores interligados que encaminham pacotes entre redes.

1.2 Classificação das redes por abrangência geográfica

Quanto à abrangência, as redes costumam ser classificadas em **PAN** (alcance pessoal, como Bluetooth), **LAN** (uma sala, prédio ou campus, como Ethernet e Wi-Fi), **MAN** (uma cidade) e **WAN** (países e continentes, como os enlaces de operadoras e os cabos submarinos) [2].

1.3 Comutação de circuitos e comutação de pacotes

Existem duas grandes formas de mover dados pelo núcleo da rede:

- **Comutação de circuitos**: antes da comunicação começar, reserva-se um caminho fim a fim com recursos dedicados (uma fração da banda de cada enlace). A rede de telefonia tradicional funcionava assim. A vantagem é o desempenho garantido; a desvantagem é o desperdício, porque os recursos ficam reservados mesmo quando ninguém está falando.
- **Comutação de pacotes**: a mensagem é quebrada em pedaços menores (pacotes), e cada pacote disputa os enlaces com os pacotes de outros usuários. Não há reserva. Os roteadores usam a técnica de **armazenar e reenviar** (*store-and-forward*): recebem o pacote inteiro, guardam num buffer e só então o transmitem no próximo enlace. É o modelo da Internet.

> **Analogia para fixar**
>
> Comutação de circuitos é como reservar uma mesa num restaurante: ela fica sua durante o horário combinado, mesmo se você se atrasar ou sair mais cedo. Comutação de pacotes é como uma praça de alimentação: não há reserva, você usa a mesa que estiver livre. Na maior parte do tempo funciona muito bem e aproveita melhor o espaço, mas no horário de pico você pode ter que esperar (isso é a **fila** nos roteadores) ou até desistir (isso é a **perda de pacotes**).

A grande vantagem da comutação de pacotes é o **ganho estatístico de multiplexação**. Considere um enlace de 1 Mbps compartilhado por usuários que transmitem a 100 kbps, mas que ficam ativos apenas 10% do tempo. Com comutação de circuitos, cabem exatamente 10 usuários. Com comutação de pacotes, é possível atender 35 usuários, e a probabilidade de mais de 10 estarem ativos ao mesmo tempo é inferior a 0,0004 [1]. Ou seja, atende-se 3,5 vezes mais gente com risco quase nulo de congestionamento.

1.4 Métricas de desempenho: atraso, perda e vazão

Para analisar quantitativamente uma rede (competência do Art. 5º, inciso V da DCN), usamos três grandezas: **atraso** (quanto tempo o pacote leva), **perda** (quantos pacotes são descartados) e **vazão** (quantos bits por segundo chegam de fato ao destino).

1.4.1 Os quatro componentes do atraso nodal

Cada vez que um pacote passa por um roteador, ele sofre quatro tipos de atraso [1]:

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{fila}} + d_{\text{trans}} + d_{\text{prop}} \tag{1}$$

- d_{proc} – **atraso de processamento**: tempo para examinar o cabeçalho, verificar erros e decidir por qual saída o pacote vai. Normalmente da ordem de microssegundos.
- d_{fila} – **atraso de fila**: tempo que o pacote espera no buffer até chegar a sua vez de ser transmitido. É o único componente que varia muito, dependendo do tráfego.
- d_{trans} – **atraso de transmissão**: tempo para "empurrar" todos os bits do pacote para o enlace. Depende do tamanho do pacote L (em bits) e da taxa do enlace R (em bits/s): $d_{\text{trans}} = L/R$.
- d_{prop} – **atraso de propagação**: tempo que um bit leva para viajar de uma ponta à outra do enlace. Depende da distância d e da velocidade de propagação no meio s (entre 2×10^8 e 3×10^8 m/s): $d_{\text{prop}} = d/s$.

> **Confusão clássica: transmissão × propagação**
>
> O atraso de transmissão **não tem nada a ver com a distância**; ele depende só do tamanho do pacote e da velocidade do enlace. Já o atraso de propagação **não tem nada a ver com o tamanho do pacote**; ele depende só da distância e do meio. Pense num pedágio: o tempo para todos os carros de uma caravana passarem pela cancela é a transmissão; o tempo que cada carro leva para percorrer a estrada até o próximo pedágio é a propagação.

1.4.2 Atraso de fila e intensidade de tráfego

Se chegam, em média, a pacotes por segundo a uma fila, cada um com L bits, e o enlace de saída transmite a R bits/s, a razão

$$\rho = \frac{L \cdot a}{R} \tag{2}$$

é chamada de **intensidade de tráfego**. Quando $\rho > 1$, chegam mais bits do que o enlace consegue enviar e a fila cresce sem limite. Mesmo com ρ próximo de 1, o atraso médio explode: no modelo de fila M/M/1, ele é proporcional a $\rho/(1-\rho)$, que tende ao infinito quando $\rho \to 1$ [1].

Por isso, projetistas de rede nunca dimensionam enlaces para operar próximos de 100% de ocupação: uma utilização média de 50% a 70% costuma ser um bom compromisso entre custo e atraso.

1.4.3 Vazão e gargalo

A **vazão** fim a fim é limitada pelo enlace mais lento do caminho, chamado de **gargalo**. Se um servidor envia a $R_s = 100$ Mbps e o cliente recebe por um enlace de $R_c = 20$ Mbps, a vazão máxima será $\min(R_s, R_c) = 20$ Mbps. Em geral, para N enlaces em série:

$$\text{Vazão} = \min\{R_1, R_2, \ldots, R_N\} \tag{3}$$

1.4.4 Produto banda-atraso

O **produto banda-atraso** ($R \times d_{\text{prop}}$) indica quantos bits "cabem" dentro do enlace ao mesmo tempo. Num enlace de 1 Gbps com 20 ms de propagação, há $10^9 \times 0{,}02 = 2 \times 10^7$ bits (2,5 MB) viajando pelo meio simultaneamente. Esse número será importante no Capítulo 6, porque a janela do TCP precisa ser pelo menos desse tamanho para aproveitar todo o enlace.

Estudo de Caso: Cálculo do Atraso Fim a Fim

Um arquivo é dividido em pacotes de 1.500 bytes. Cada pacote atravessa **dois enlaces** em série, ligados por um roteador. Os dados são:

- Enlace 1: taxa $R_1 = 10$ Mbps, comprimento 1.000 km;
- Enlace 2: taxa $R_2 = 100$ Mbps, comprimento 3.000 km;
- Velocidade de propagação nos dois meios: $s = 2 \times 10^8$ m/s;
- Atrasos de processamento e fila desprezíveis.

Passo 1 – tamanho em bits: $L = 1500 \times 8 = 12.000$ bits.
Passo 2 – transmissão em cada enlace: $d_{\text{trans},1} = \dfrac{12.000}{10 \times 10^6} = 1{,}2$ ms e $d_{\text{trans},2} = \dfrac{12.000}{100 \times 10^6} = 0{,}12$ ms.
Passo 3 – propagação em cada enlace: $d_{\text{prop},1} = \dfrac{10^6 \text{ m}}{2 \times 10^8} = 5$ ms e $d_{\text{prop},2} = \dfrac{3 \times 10^6 \text{ m}}{2 \times 10^8} = 15$ ms.
Passo 4 – total (store-and-forward): $d_{\text{total}} = 1{,}2 + 5 + 0{,}12 + 15 = 21{,}32$ ms.
Interpretação: mesmo o enlace 2 sendo 10 vezes mais rápido, ele contribui com o maior atraso, porque é mais longo. Em redes de longa distância, a propagação costuma dominar; em redes locais, a transmissão costuma dominar.

1.5 Protocolos e arquitetura em camadas

Um **protocolo** define o formato e a ordem das mensagens trocadas entre duas ou mais entidades, e as ações tomadas no envio ou no recebimento de uma mensagem [1]. Tanenbaum [2] destaca que todo protocolo tem três elementos:

- **Sintaxe**: o formato dos dados (quais campos, em que ordem, com quantos bits);
- **Semântica**: o significado de cada campo e a ação correspondente;
- **Temporização**: quando os dados devem ser enviados e em que velocidade.

Como projetar um único protocolo para resolver todos os problemas da comunicação seria inviável, as redes são organizadas em **camadas**. Cada camada resolve um pedaço do problema, oferece **serviços** para a camada de cima e usa os serviços da camada de baixo. Essa é uma aplicação direta do princípio de **abstração**, citado explicitamente na DCN (Art. 5º, §1º, inciso X) como um tema fundamental da

Ciência da Computação.

Três conceitos precisam ficar bem separados:

- **Serviço**: *o que* a camada oferece para a camada superior (por exemplo, “entrega confiável de bytes”).
- **Interface**: *como* a camada superior acessa esse serviço (as primitivas, como `send()` e `receive()`).
- **Protocolo**: *como* a camada conversa com a camada de mesmo nível no outro equipamento (comunicação **par a par**, ou *peer-to-peer*).

> **Analogia para fixar**
>
> Imagine dois diretores de empresas, um no Brasil e outro no Japão, que querem fechar um contrato. O diretor brasileiro escreve a proposta em português e entrega ao tradutor (camada de baixo). O tradutor converte para inglês e entrega à secretária, que coloca num envelope com endereço e manda pelo correio. No Japão, o processo é invertido. Cada “camada” só conversa de verdade com a sua equivalente do outro lado: os diretores discutem o contrato, os tradutores “conversam” em inglês, as secretárias se entendem pelo formato do envelope. E dá para trocar o tradutor ou a transportadora sem que os diretores precisem mudar nada. Esse é o grande benefício das camadas: **modularidade**.

1.6 O Modelo de Referência OSI

O modelo **OSI** (*Open Systems Interconnection*) foi padronizado pela ISO no início dos anos 1980 (documento atual: ISO/IEC 7498-1) [15]. Ele divide a comunicação em **sete camadas**. É importante entender que o OSI é um **modelo de referência**: ele descreve o que cada camada deve fazer, mas os protocolos propostos junto com ele praticamente não são usados hoje. Mesmo assim, o vocabulário do OSI é usado no mercado o tempo todo (“switch de camada 2”, “balanceador de camada 7”, “problema de camada 1”).

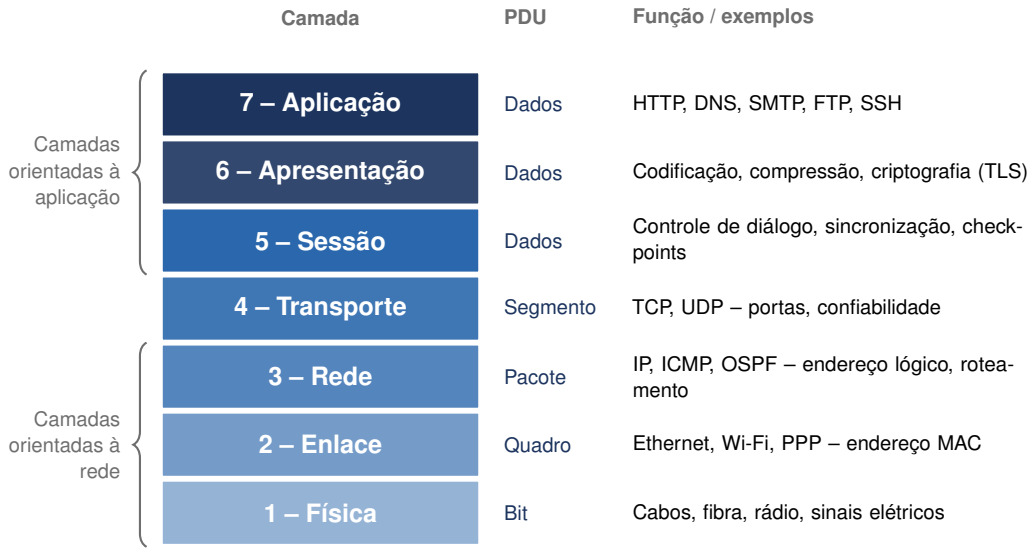


Figura 2: As sete camadas do modelo OSI, suas unidades de dados (PDU) e exemplos

Fonte: adaptado de ISO/IEC 7498-1 [15] e Tanenbaum, Feamster e Wetherall [2].

1.6.1 Função de cada camada

1. **Física**: transmite bits brutos pelo meio. Define níveis de tensão, tipo de conector, modulação, taxa de sinalização e se a transmissão é *simplex*, *half-duplex* ou *full-duplex*. Não entende o significado dos bits.
2. **Enlace de dados**: organiza os bits em **quadros**, entrega dados entre nós *vizinhos* (no mesmo enlace), detecta (e às vezes corrige) erros e controla o acesso ao meio compartilhado. Usa endereços físicos (MAC).
3. **Rede**: leva pacotes da origem ao destino atravessando várias redes. É responsável pelo **endereçamento lógico** (IP) e pelo **roteamento**.
4. **Transporte**: oferece comunicação **fim a fim** entre processos. Pode garantir entrega confiável e ordenada, controle de fluxo e de congestionamento. Aqui aparecem as **portas**.
5. **Sessão**: estabelece, gerencia e encerra sessões de diálogo; permite pontos de sincronização para retomar transferências interrompidas.
6. **Apresentação**: cuida da **sintaxe e semântica** da informação: conversão de códigos de caracteres, compressão e criptografia.
7. **Aplicação**: contém os protocolos usados diretamente pelos programas do usuário.

1.6.2 Comunicação vertical e horizontal

No OSI, os dados descem pelas camadas na origem, atravessam o meio físico e sobem pelas camadas no destino (comunicação **vertical**, real). Ao mesmo tempo, cada camada "conversa" logicamente com sua par no outro lado (comunicação **horizontal**, virtual), por meio dos cabeçalhos que ela acrescenta. Os equipamentos intermediários só processam as camadas de que precisam: um switch vai até a camada 2, um roteador até a camada 3.

1.6.3 Por que o OSI não "venceu"?

Quando o OSI ficou pronto, o TCP/IP já estava difundido, e as camadas de sessão e apresentação acabaram quase vazias [2]. Mesmo assim, o modelo continua sendo a principal ferramenta didática e de diagnóstico: quando um técnico diz que "o problema é na camada 1", todo mundo entende que é cabo, conector ou sinal.

Questão comentada (estilo ENADE)

Um técnico percebe que dois computadores da mesma sala não se comunicam. O LED da placa de rede de um deles está apagado. Em seguida, após trocar o cabo, os computadores se enxergam, mas o navegador ainda não abre um site pelo nome, apenas pelo endereço IP. Assinale a alternativa que indica, respectivamente, as camadas OSI dos dois problemas.
(A) Enlace e Rede **(B)** Física e Aplicação **(C)** Física e Transporte **(D)** Rede e Sessão **(E)** Enlace e Apresentação
Resposta: (B). O LED apagado e a solução pela troca do cabo indicam falha na camada **física** (sinal/meio). Já conseguir acessar pelo IP mas não pelo nome indica falha na resolução de nomes, ou seja, no **DNS**, que é um protocolo da camada de **aplicação**. A alternativa (A) é uma pegadinha comum: o cabo não é "enlace", e sim meio físico.

2 Modelo TCP/IP e Encapsulamento de Dados

Se o OSI é o modelo usado para *ensinar* e *diagnosticar*, o **TCP/IP** é o modelo que de fato *roda* na Internet. Ele nasceu da ARPANET, rede financiada pelo Departamento de Defesa dos Estados Unidos no fim dos anos 1960. Em 1974, Vinton Cerf e Robert Kahn publicaram a proposta de um protocolo capaz de interligar redes heterogêneas, que depois foi dividido em dois: o **TCP** (transporte) e o **IP** (rede) [1, 2]. Em 1º de janeiro de 1983, a ARPANET migrou oficialmente para o TCP/IP, data considerada por muitos como o "nascimento" da Internet moderna.

Um dos objetivos de projeto era que a rede continuasse funcionando mesmo se parte dela fosse destruída. Por isso, o TCP/IP escolheu uma camada de rede **sem conexão** (cada pacote é roteado de forma independente) e colocou a inteligência nas pontas (nos hosts), e não no núcleo. Esse é o famoso **princípio fim a fim** (*end-to-end argument*).

2.1 As camadas do modelo TCP/IP

A pilha TCP/IP é descrita originalmente com **quatro camadas** (RFC 1122). Muitos livros, como Kurose e Ross [1] e Forouzan [3], usam uma versão "híbrida" com **cinco camadas**, separando a camada de acesso à rede em física e enlace. As duas visões são corretas; o importante é saber relacionar cada uma com o OSI.

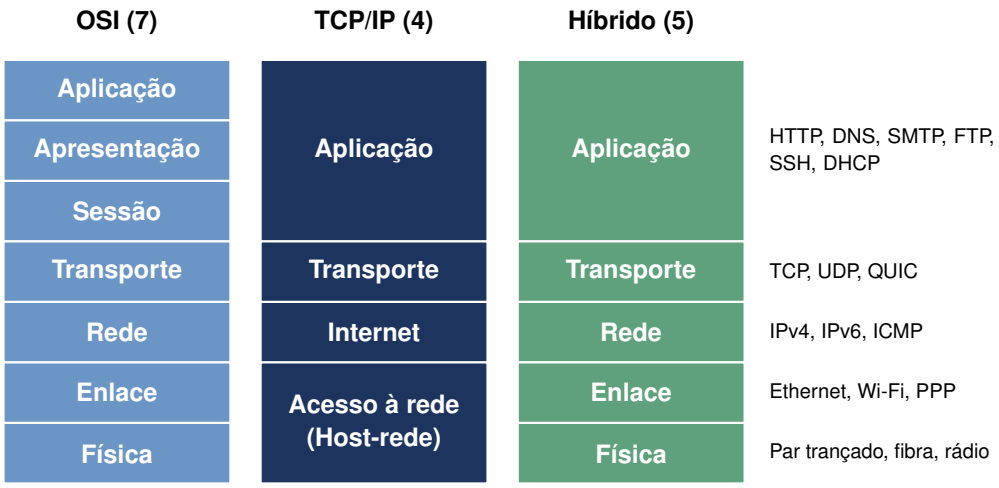


Figura 3: Correspondência entre o modelo OSI, o modelo TCP/IP original e o modelo híbrido de cinco camadas

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2] e Kurose e Ross [1].

Resumindo as funções de cada camada do TCP/IP:

- **Aplicação**: engloba as funções de aplicação, apresentação e sessão do OSI. Se a aplicação precisar de criptografia ou compressão, ela mesma (ou uma biblioteca como o TLS) resolve.
- **Transporte**: comunicação fim a fim entre processos, identificados por **portas**. Os dois protocolos principais são o TCP (confiável, orientado a conexão) e o UDP (simples, sem conexão).
- **Internet (Rede)**: coração da arquitetura. O IP entrega datagramas entre hosts de redes diferentes, no modelo de **melhor esforço** (*best effort*): tenta entregar, mas não garante ordem, ausência de duplicatas nem a própria entrega.
- **Acesso à rede**: o TCP/IP original praticamente não especifica essa camada; diz apenas que o host deve se conectar à rede usando algum protocolo capaz de transportar pacotes IP. Isso permitiu que o IP rodasse sobre qualquer tecnologia: Ethernet, Wi-Fi, 4G, fibra, satélite...

2.2 O modelo “ampulheta”

Uma forma elegante de enxergar a arquitetura da Internet é o modelo da **ampulheta** (*hourglass*) [4]. Em cima, há uma enorme variedade de aplicações; embaixo, uma enorme variedade de tecnologias físicas. No meio, há um único ponto de convergência: o protocolo IP. Qualquer aplicação nova funciona em qualquer rede física, desde que ambas “falem” IP. Essa é a razão do sucesso da Internet.

Vale guardar duas diferenças: no OSI o modelo veio antes dos protocolos, e no TCP/IP os protocolos vieram antes do modelo; além disso, a camada de rede do TCP/IP é apenas sem conexão (IP), enquanto o OSI previa as duas formas [2].

2.3 Encapsulamento de dados

Encapsulamento é o processo em que cada camada, ao receber dados da camada superior, acrescenta suas próprias informações de controle (um **cabeçalho** e, em alguns casos, um **trailer**) e repassa o conjunto para a camada inferior [1, 3]. Os dados recebidos da camada de cima são chamados de **carga útil** (*payload*) daquela camada. O conjunto “cabeçalho + carga útil” forma a **PDU** (*Protocol Data Unit*) da camada.

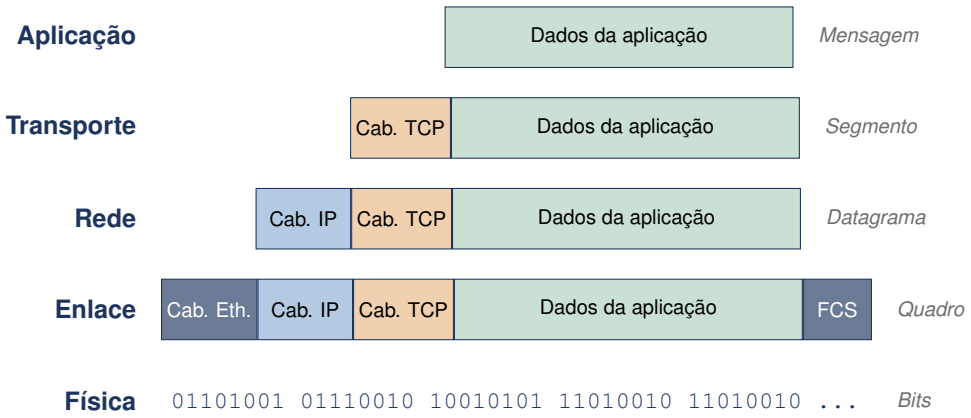


Figura 4: Encapsulamento de uma requisição HTTP ao descer pela pilha TCP/IP

Fonte: adaptado de Kurose e Ross [1] e Forouzan [3].

Observe três detalhes importantes:

1. Cada camada **trata tudo o que veio de cima como dados opacos**. O IP não sabe (e não precisa saber) que dentro do segmento há uma requisição HTTP.
2. A camada de enlace é a única que costuma acrescentar um **trailer**: o FCS (*Frame Check Sequence*), um código CRC usado para detectar erros de transmissão (veja o Capítulo 5).
3. A camada física não adiciona cabeçalho no sentido lógico, mas acrescenta sinais de sincronização (no Ethernet, o **preâmbulo** de 7 bytes e o delimitador de início de quadro de 1 byte).

2.3.1 Desencapsulamento e multiplexação

No destino, o processo é invertido: cada camada lê e remove o seu cabeçalho e entrega a carga útil à camada de cima. Para saber **para quem** entregar, cada cabeçalho carrega um campo que identifica o protocolo superior: o *EtherType* no Ethernet (`0x0800` = IPv4, `0x86DD` = IPv6), o campo *Protocolo* no IP (6 = TCP, 17 = UDP, 1 = ICMP) e a *porta de destino* no transporte (443 = HTTPS, 53 = DNS) [4].

2.3.2 Endereços em cada camada

Um ponto que costuma gerar confusão é a existência de "vários endereços". Cada camada tem o seu, com escopo diferente:

Tabela 3: Tipos de endereço na pilha TCP/IP

Camada	Endereço	Tamanho	O que identifica / escopo
Aplicação	Nome (URL, e-mail)	Variável	Recurso ou usuário, legível por humanos
Transporte	Porta	16 bits	Processo (aplicação) dentro do host
Rede	Endereço IP	32 bits (v4) / 128 (v6)	Interface do host na Internet; fim a fim
Enlace	Endereço MAC	48 bits	Placa de rede; válido só no enlace local (salto a salto)

Fonte: adaptado de Forouzan [3].

Analogia para fixar

Pense numa encomenda enviada para um apartamento. O **endereço IP** é o endereço do prédio (rua, número, cidade): ele não muda durante toda a viagem. O **endereço MAC** é como a etiqueta que cada transportadora coloca para o trecho que ela vai fazer (do centro de distribuição A para o B): a cada trecho a etiqueta é trocada. E a **porta** é o número do apartamento: depois que a encomenda chega ao prédio, é ela que diz para qual morador entregar. Por isso se diz que o IP é "fim a fim" e o MAC é "salto a salto".

2.4 Eficiência e sobrecarga (overhead)

Todo cabeçalho é informação de controle, e não dado útil. A razão entre os dados úteis e o total transmitido mede a **eficiência** do encapsulamento:

$$\eta = \frac{\text{dados úteis}}{\text{dados úteis} + \text{cabeçalhos} + \text{trailers}} \tag{4}$$

Estudo de Caso: Eficiência de um Quadro Ethernet com TCP/IPv4

Uma aplicação envia dados por TCP sobre IPv4 e Ethernet, com o tamanho máximo de carga útil do Ethernet (MTU) de 1.500 bytes. Considere cabeçalhos mínimos (sem opções).

Passo 1 – quanto sobra para a aplicação: a MTU de 1.500 bytes precisa comportar o cabeçalho IPv4 (20 bytes) e o TCP (20 bytes). Sobram $1500 - 40 = 1460$ bytes de dados. Esse valor é o famoso **MSS** (*Maximum Segment Size*) padrão.

Passo 2 – o que vai de fato para o fio: cabeçalho Ethernet (14) + MTU (1.500) + FCS (4) = 1.518 bytes de quadro. Somando preâmbulo e SFD (8) e o intervalo mínimo entre quadros (12 bytes de "silêncio"), cada quadro ocupa o equivalente a 1.538 bytes.

Passo 3 – eficiência:

$$\eta = \frac{1460}{1538} \approx 0{,}949 \quad (94{,}9\%)$$

Passo 4 – impacto na vazão: num enlace de 1 Gbps, a vazão máxima de dados de aplicação é aproximadamente $0{,}949 \times 1000 \approx 949$ Mbps.

E se a aplicação enviar só 1 byte (uma tecla digitada numa sessão SSH, por exemplo)? O quadro terá o tamanho mínimo do Ethernet (64 bytes, mais 20 de preâmbulo e intervalo = 84), e a eficiência cai para $1/84 \approx 1{,}2\%$. Por isso existem técnicas como o **algoritmo de Nagle**, que agrupa pequenos envios num único segmento.

2.5 Acompanhando um pacote de ponta a ponta

Para juntar tudo, vale acompanhar o que acontece quando um aluno, conectado ao Wi-Fi da faculdade, digita `www.uscs.edu.br` no navegador. De forma simplificada:

1. O computador ainda não tem endereço IP, então usa o **DHCP** (aplicação, sobre UDP) para obter IP, máscara, gateway padrão e servidor DNS.
2. Para enviar qualquer coisa ao gateway, precisa do MAC dele: usa o **ARP** ("quem tem o IP do gateway?").
3. O navegador pergunta ao servidor **DNS** qual é o IP de `www.uscs.edu.br` (consulta UDP, porta 53).
4. Com o IP em mãos, abre uma conexão **TCP** (apresentação em três vias) na porta 443 e negocia o **TLS**.
5. Envia a requisição **HTTP** `GET`. Ela é encapsulada em segmento TCP, datagrama IP e quadro Wi-Fi e, a cada salto, é reencapsulada num novo quadro (Ethernet, fibra etc.) até chegar ao servidor.
6. O servidor desencapsula, processa e devolve a página pelo caminho inverso.

Cada um desses passos é detalhado nos próximos capítulos. Esse exemplo, conhecido como "um dia na vida de uma requisição web", é uma das melhores formas de revisar a pilha inteira [1].

Questão comentada (estilo ENADE)

Uma aplicação envia 200 bytes de dados usando UDP sobre IPv4 sem opções, em um enlace Ethernet. Considerando os cabeçalhos de transporte, de rede e de enlace (14 bytes), além do trailer FCS (4 bytes), o tamanho do quadro e a porcentagem de sobrecarga são, aproximadamente: **(A)** 228 bytes e 12,3% **(B)** 246 bytes e 18,7% **(C)** 242 bytes e 17,4% **(D)** 258 bytes e 22,5% **(E)** 238 bytes e 16,0%

Resposta: (B). O UDP tem cabeçalho de 8 bytes e o IPv4, de 20 bytes. Quadro: $14 + 20 + 8 + 200 + 4 = 246$ bytes. Sobrecarga: $46/246 \approx 18{,}7\%$. A alternativa (C) esquece o FCS; a (A) considera apenas os cabeçalhos de rede e transporte; a (D) usaria cabeçalho TCP no lugar do UDP.

3 Meios de Transmissão, Topologias e Equipamentos de Interconexão

Todo dado que trafega numa rede, em algum momento, vira um fenômeno físico: uma variação de tensão num fio de cobre, um pulso de luz numa fibra ou uma onda eletromagnética no ar. Este capítulo trata dessa base física, de como os equipamentos são arranjados (topologia) e de quais dispositivos são usados para interligá-los.

3.1 Conceitos básicos de transmissão

Antes de comparar os meios, é preciso fixar alguns conceitos [2, 3]:

- **Largura de banda (analógica)**: faixa de frequências, em hertz (Hz), que o meio consegue transmitir sem atenuação significativa.
- **Taxa de transmissão (ou taxa de bits)**: quantidade de bits enviados por segundo (bps). No dia a dia, "largura de banda" também é usada com esse sentido ("minha banda é de 500 Mbps").
- **Atenuação**: perda de potência do sinal com a distância, medida em decibéis (dB).

- **Ruído**: sinais indesejados que se somam ao sinal original (ruído térmico, interferência eletromagnética, diafonia ou *crosstalk*).
- **Modos de transmissão**: *simplex* (só um sentido, como uma transmissão de TV), *half-duplex* (os dois sentidos, mas um de cada vez, como um walkie-talkie) e *full-duplex* (os dois sentidos simultaneamente, como uma ligação telefônica).

3.1.1 Decibéis e atenuação

O decibel expressa a relação entre duas potências numa escala logarítmica:

$$\text{dB} = 10 \log_{10} \left(\frac{P_2}{P_1} \right) \tag{5}$$

Uma perda de 3 dB significa que o sinal caiu para metade da potência; uma perda de 10 dB, para um décimo; e de 20 dB, para um centésimo. A vantagem da escala em dB é que as perdas e ganhos de vários trechos simplesmente se **somam**. Se um cabo perde 2 dB, um conector perde 0,5 dB e um amplificador ganha 10 dB, o saldo é $-2 - 0{,}5 + 10 = +7{,}5$ dB.

3.1.2 Os limites teóricos: Nyquist e Shannon

Dois resultados clássicos estabelecem o máximo de bits que um canal pode transportar. Eles são um ótimo exemplo da competência "conhecer os limites da computação" (Art. 5º, II, da DCN).

Teorema de Nyquist (canal sem ruído): para um canal com largura de banda B (Hz) e sinal com V níveis discretos,

$$C_{\text{Nyquist}} = 2B \log_2 V \quad \text{bits/s} \tag{6}$$

Teorema de Shannon (canal com ruído): para um canal com largura de banda B e relação sinal-ruído S/N (em escala linear, não em dB),

$$C_{\text{Shannon}} = B \log_2 \left(1 + \frac{S}{N} \right) \quad \text{bits/s} \tag{7}$$

A diferença é importante: Nyquist diz que dá para aumentar a taxa usando mais níveis de sinal; Shannon mostra que, na presença de ruído, existe um teto que **nenhuma** técnica de modulação consegue ultrapassar.

Estudo de Caso: Capacidade de uma Linha Telefônica

Uma linha telefônica analógica tem largura de banda de aproximadamente 3.000 Hz e relação sinal-ruído de 30 dB. Qual a capacidade máxima teórica?
Passo 1 – converter S/N de dB para escala linear: $30 = 10 \log_{10}(S/N) \Rightarrow S/N = 10^3 = 1000$.
Passo 2 – aplicar Shannon: $C = 3000 \times \log_2(1 + 1000) \approx 3000 \times 9{,}97 \approx 29{,}9$ kbps.
Passo 3 – quantos níveis de sinal seriam necessários (Nyquist)? $29.900 = 2 \times 3000 \times \log_2 V \Rightarrow \log_2 V \approx 4{,}98 \Rightarrow V \approx 32$ níveis.
Interpretação: por mais sofisticado que seja o modem, não há como passar de uns 30 kbps numa linha com essas características. É por isso que o acesso discado ficou "travado" na casa dos 33,6 kbps, e a evolução para velocidades maiores exigiu outras tecnologias (DSL, cabo, fibra), que usam uma largura de banda muito maior do mesmo par de fios ou de outro meio [2].

3.2 Meios guiados

Nos meios **guiados**, o sinal se propaga confinado num meio sólido.

3.2.1 Par trançado

É formado por pares de fios de cobre isolados e trançados entre si. O **trançamento** faz com que as interferências captadas pelos dois fios de um par sejam praticamente iguais e se cancelem na recepção (transmissão diferencial). Existem versões sem blindagem (UTP) e com blindagem (STP, FTP, S/FTP). É o meio mais usado em redes locais, com conector RJ-45 e alcance padrão de 100 metros por segmento.

Tabela 4: Categorias de cabos de par trançado

Categoria	Frequência	Taxa típica	Observações
Cat 5e	100 MHz	1 Gbps (100 m)	Ainda muito comum em instalações antigas
Cat 6	250 MHz	1 Gbps (100 m); 10 Gbps (até 55 m)	Separador interno reduz diafonia
Cat 6A	500 MHz	10 Gbps (100 m)	Padrão recomendado para instalações novas
Cat 8	2.000 MHz	25/40 Gbps (até 30 m)	Uso em data centers, ligações curtas

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2] e das normas ANSI/TIA-568.

3.2.2 Cabo coaxial

Condutor central envolto por isolante e malha metálica, com boa blindagem. Hoje é usado principalmente em TV a cabo e acesso à Internet via cabo (padrão DOCSIS).

3.2.3 Fibra óptica

A fibra transmite **pulsos de luz** por um núcleo de vidro ou plástico, envolvido por uma casca de índice de refração menor. A luz fica "presa" no núcleo por **reflexão interna total**. Principais vantagens: altíssima largura de banda, baixa atenuação (cerca de 0,2 dB/km na janela de 1.550 nm), imunidade a interferência eletromagnética, dificuldade de "grampear" e peso reduzido.

- **Multimodo**: núcleo mais largo (50 ou 62,5 µm), a luz percorre vários caminhos (modos). Isso causa **dispersão modal**, limitando a distância a centenas de metros. Usa LEDs ou lasers VCSEL, mais baratos. Ideal para dentro de prédios e data centers.
- **Monomodo**: núcleo muito fino (cerca de 9 µm), a luz segue praticamente um único caminho. Alcança dezenas ou centenas de quilômetros. Usa laser. É a fibra dos backbones e do FTTH (fibra até a casa).

3.3 Meios não guiados (sem fio)

Nos meios **não guiados**, o sinal se propaga pelo espaço como onda eletromagnética. O comportamento depende muito da frequência:

Uma regra prática: **quanto maior a frequência, maior a capacidade e menor o alcance**. Frequências baixas contornam obstáculos e vão longe, mas têm pouca banda disponível; frequências altas carregam muitos dados, mas são facilmente bloqueadas. O Capítulo 9 retoma esse ponto ao falar de Wi-Fi e 5G.

3.4 Topologias de rede

A **topologia** descreve como os nós estão arranjados e ligados. Distingue-se a topologia **física** (como os cabos estão de fato instalados) da **lógica** (como os dados circulam). Uma rede com hub, por exemplo, é fisicamente uma estrela, mas logicamente um barramento, porque tudo que um computador envia chega a todos os outros.

Tabela 5: Faixas do espectro usadas em comunicação sem fio

Tipo	Faixa aproximada	Características e usos
Rádio	3 kHz a 1 GHz	Onidirecional, atravessa paredes; rádio FM, TV, celular em faixas baixas (700 MHz)
Micro-ondas	1 GHz a 300 GHz	Mais direcional; Wi-Fi (2,4/5/6 GHz), 4G/5G, enlaces ponto a ponto, satélite
Ondas milimétricas	24 GHz a 100 GHz	Altíssima capacidade, mas curto alcance e bloqueio por obstáculos (5G mmWave)
Infravermelho	300 GHz a 400 THz	Não atravessa paredes; controles remotos, comunicação de curta distância

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2] e Forouzan [3].

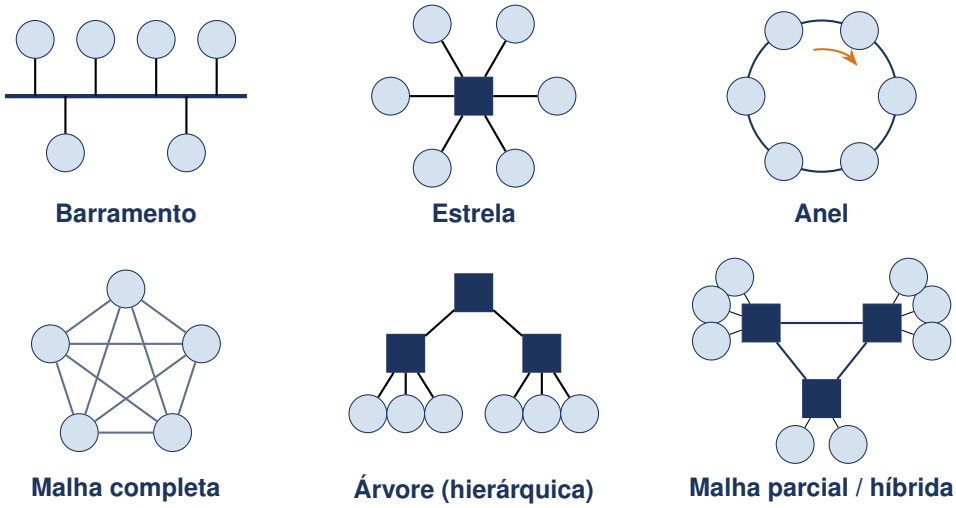


Figura 5: Principais topologias físicas (quadrados = equipamentos de interconexão; círculos = hosts)

Fonte: adaptado de Forouzan [3] e Comer [4].

Tabela 6: Vantagens e desvantagens das topologias

Topologia	Vantagens	Desvantagens
Barramento	Barata, pouco cabo, fácil de instalar	Um rompimento derruba tudo; colisões; difícil diagnóstico
Estrela	Falha de um cabo afeta só um nó; fácil de gerenciar e expandir	Ponto único de falha no equipamento central
Anel	Acesso ordenado (sem colisões, ex.: token ring); desempenho previsível	Falha de um nó pode interromper o anel (resolvido com anel duplo)
Malha completa	Máxima redundância e desempenho	Custo altíssimo: número de enlaces cresce com n^2
Árvore	Escalável, reflete a hierarquia da organização	Falha num nó superior isola toda a subárvore
Híbrida	Combina o melhor de cada topologia; é a realidade da Internet	Projeto e gerenciamento mais complexos

Fonte: adaptado de Forouzan [3].

Na **malha completa** com n nós, cada nó se liga a todos os outros $n-1$. Como cada enlace é contado duas vezes, o número de enlaces é:

$$E = \frac{n(n-1)}{2} \tag{8}$$

Com 5 nós são 10 enlaces; com 50 nós, 1.225; com 1.000 nós, quase meio milhão. Esse crescimento

quadrático ($\Theta(n^2)$) explica por que malhas completas só são usadas entre poucos nós críticos, como os roteadores de núcleo de uma operadora.

> **Analogia para fixar**
>
> A topologia estrela é como um condomínio com portaria central: toda encomenda passa pela portaria. Se um apartamento tem problema, os outros continuam recebendo normalmente; mas se a portaria fecha, ninguém recebe nada. Já a malha completa é como se cada apartamento tivesse uma porta direta para todos os outros: muito resiliente, mas imagine quantas portas um prédio de 100 apartamentos precisaria.

3.5 Equipamentos de interconexão

Os equipamentos de rede são classificados pela camada mais alta que eles "entendem".

3.5.1 Repetidor e hub (camada 1)

O **repetidor** regenera o sinal para aumentar o alcance. O **hub** é um repetidor com várias portas: tudo que chega numa porta é copiado para todas as outras. Ele não entende quadros nem endereços, só bits. Consequência: todos os dispositivos ligados a um hub estão no mesmo **domínio de colisão** e operam em *half-duplex*. Hubs estão praticamente extintos.

3.5.2 Bridge e switch (camada 2)

O **switch** (comutador) lê o endereço MAC de destino de cada quadro e o encaminha **apenas pela porta certa**. Para isso, mantém uma **tabela de endereços MAC** (tabela CAM), construída por **autoaprendizado**:

1. Quando um quadro chega pela porta p, o switch anota: "o MAC de origem deste quadro está na porta p".
2. Se o MAC de destino está na tabela, o quadro é encaminhado só para aquela porta (**filtragem**).
3. Se o destino é desconhecido (ou é broadcast `FF:FF:FF:FF:FF:FF`), o quadro é enviado para todas as portas, menos a de origem (**inundação**, ou *flooding*).
4. As entradas expiram após um tempo sem uso (tipicamente 300 segundos).

Cada porta de um switch é um domínio de colisão separado e pode operar em *full-duplex*. Porém, o switch repassa broadcasts para todas as portas, então todas continuam no mesmo **domínio de broadcast** (a menos que se usem VLANs, vistas no Capítulo 5).

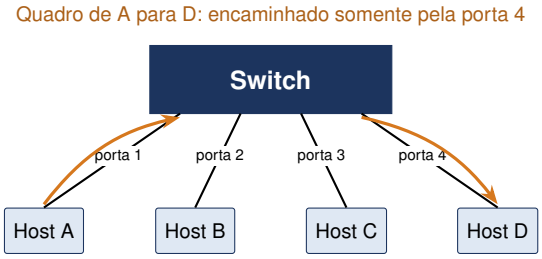


Tabela MAC aprendida

MAC	Porta
MAC de A	1
MAC de B	2
MAC de D	4

Figura 6: Encaminhamento seletivo com base na tabela de endereços MAC do switch

Fonte: adaptado de Kurose e Ross [1].

3.5.3 Roteador (camada 3)

O **roteador** interliga **redes diferentes**. Ele lê o endereço IP de destino, consulta sua **tabela de roteamento** e decide para qual próximo salto enviar o pacote. Roteadores **não repassam broadcasts** de camada 2; cada interface de um roteador define um domínio de broadcast separado. Também decrementam o campo TTL do IP e podem fazer NAT, filtragem de pacotes e QoS.

Completam a lista o **switch de camada 3** (roteia entre VLANs em hardware), o **ponto de acesso** (ponte entre Wi-Fi e rede cabeada), o **proxy** (atua na camada 7), o **firewall** (Capítulo 8) e o **modem/ONT**, que converte o sinal para o meio de acesso da operadora.

Tabela 7: Resumo dos equipamentos de interconexão

Equipamento	Camada	Decide com base em	Separa colisão?	Separa broadcast?
Repetidor / Hub	1	Nada (repete bits)	Não	Não
Bridge / Switch	2	Endereço MAC	Sim (por porta)	Não (sem VLAN)
Roteador	3	Endereço IP	Sim	Sim (por interface)
Gateway / Proxy	4 a 7	Dados de aplicação	Sim	Sim

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2] e Forouzan [3].

Estudo de Caso: Contando Domínios de Colisão e de Broadcast

Uma empresa tem a seguinte rede: um **roteador** com duas interfaces LAN. Na interface 1 está ligado um **switch** de 8 portas, com 5 computadores e um **hub** (o hub tem 4 computadores ligados). Na interface 2 está ligado outro **switch** com 6 computadores. Quantos domínios de colisão e de broadcast existem?
Domínios de broadcast: cada interface do roteador delimita um domínio. Logo, **2 domínios de broadcast**.
Domínios de colisão: cada porta de switch em uso é um domínio.

- Switch 1: 5 portas com PCs + 1 porta com o hub + 1 porta ligada ao roteador = 7 domínios (o hub e seus 4 PCs formam um único domínio, porque o hub não separa colisões).
- Switch 2: 6 portas com PCs + 1 porta ligada ao roteador = 7 domínios.

Total: $7 + 7 =$ **14 domínios de colisão**.
Observação: em enlaces *full-duplex* (switch-PC), colisões não ocorrem na prática, mas a contagem de domínios continua sendo cobrada dessa forma em provas.

Questão comentada (estilo ENADE)

Uma rede local com 40 computadores ligados a hubs apresenta lentidão nos horários de pico. O administrador substitui os hubs por switches, sem alterar mais nada. Sobre o efeito dessa troca, é correto afirmar que:
(A) o número de domínios de broadcast aumenta, reduzindo o tráfego de ARP. **(B)** o número de domínios de colisão aumenta, e cada porta passa a poder operar em *full-duplex*. **(C)** os computadores passam a estar em redes IP diferentes. **(D)** o tráfego de broadcast deixa de ser propagado. **(E)** não há mudança de desempenho, pois ambos operam na camada 2.
Resposta: (B). O switch cria um domínio de colisão por porta e permite *full-duplex*, eliminando colisões. Ele **não** separa broadcast (A e D erradas), não altera o endereçamento IP (C errada), e o hub opera na camada 1, não na 2 (E errada).

4 Endereçamento IP, Máscaras de Sub-rede e IPv6

O protocolo IP (*Internet Protocol*) é a “cola” da Internet. Ele define um formato de pacote comum e um esquema de endereçamento global que permite identificar qualquer interface conectada à rede. Este capítulo cobre o IPv4 (RFC 791), a divisão em sub-redes, o NAT e o IPv6 (RFC 8200), que é a solução definitiva para o esgotamento de endereços.

4.1 O datagrama IPv4

O cabeçalho IPv4 tem 20 bytes (sem opções) e é desenhado em linhas de 32 bits:

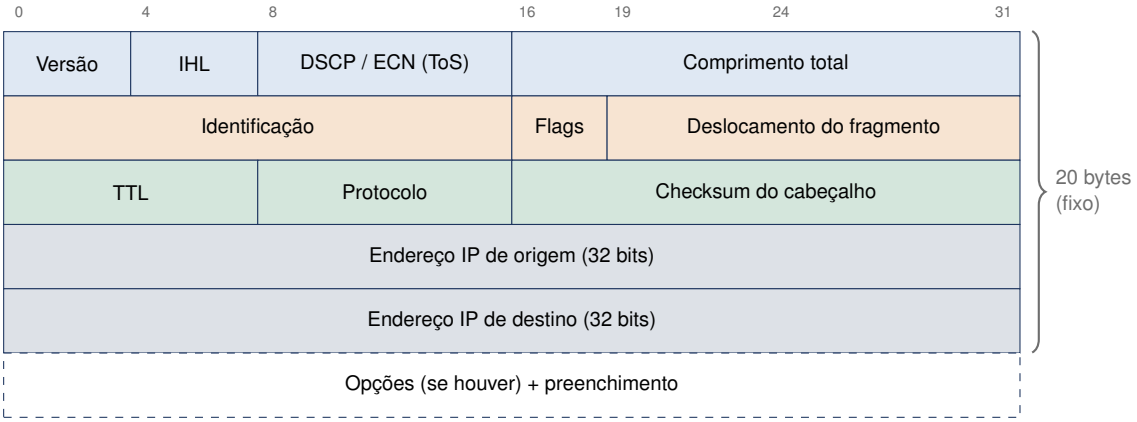


Figura 7: Formato do cabeçalho IPv4

Fonte: adaptado da RFC 791 [10] e de Kurose e Ross [1].

Campos que mais aparecem em provas e no dia a dia:

- **Versão**: 4 para IPv4.
- **IHL**: tamanho do cabeçalho em palavras de 32 bits (mínimo 5, ou seja, 20 bytes).
- **Comprimento total**: tamanho do datagrama inteiro (cabeçalho + dados), no máximo 65.535 bytes.
- **Identificação, flags e deslocamento**: usados na **fragmentação**. Quando um datagrama é maior que a MTU do próximo enlace, ele é dividido em fragmentos, que só são remontados no destino. A flag DF (*Don’t Fragment*) proíbe a fragmentação; MF (*More Fragments*) indica que ainda há fragmentos. O deslocamento é medido em unidades de 8 bytes.
- **TTL** (*Time To Live*): decrementado em 1 a cada roteador. Ao chegar a zero, o pacote é descartado e uma mensagem ICMP “tempo excedido” volta para a origem. Isso impede que pacotes fiquem circulando para sempre em laços de roteamento. O comando `traceroute` explora exatamente esse comportamento.
- **Protocolo**: indica quem está dentro (6 = TCP, 17 = UDP, 1 = ICMP).
- **Checksum**: protege apenas o cabeçalho; é recalculado a cada salto, pois o TTL muda.

4.2 Estrutura do endereço IPv4

Um endereço IPv4 tem **32 bits**, escritos em **notação decimal pontuada**: quatro octetos (bytes), cada um de 0 a 255, separados por pontos. Isso dá $2^{32} \approx 4{,}3$ bilhões de endereços.

Todo endereço é dividido em duas partes: a **parte de rede** (igual para todos os hosts da mesma rede) e a **parte de host** (que identifica a interface dentro da rede). Quem define onde uma termina e a outra começa é a **máscara**.

4.2.1 Endereçamento por classes (histórico)

Até 1993, a divisão rede/host era fixa, determinada pelos primeiros bits do endereço:

Tabela 8: Classes de endereços IPv4

Classe	Bits iniciais	Faixa do 1º octeto	Máscara padrão	Uso
A	0	1 a 126	/8 (255.0.0.0)	Redes muito grandes (16,7 milhões de hosts)
B	10	128 a 191	/16 (255.255.0.0)	Redes médias (65.534 hosts)
C	110	192 a 223	/24 (255.255.255.0)	Redes pequenas (254 hosts)
D	1110	224 a 239	–	Multicast
E	1111	240 a 255	–	Reservado / experimental

Fonte: adaptado de Comer [4] e da RFC 791 [10].

O problema das classes era o desperdício: uma empresa com 300 hosts não cabia numa classe C (254 hosts) e recebia uma classe B inteira (65.534 hosts), jogando fora mais de 65 mil endereços. A faixa 127.x.x.x é reservada para *loopback*.

4.3 Máscara de sub-rede e CIDR

A **máscara de sub-rede** é um número de 32 bits em que os bits “1” marcam a parte de rede e os bits “0” marcam a parte de host. Com o **CIDR** (*Classless Inter-Domain Routing*, RFC 4632), a fronteira pode ficar em qualquer posição, e a máscara passou a ser escrita de forma compacta como /**n**, em que n é a quantidade de bits de rede.

Tabela 9: Máscaras mais comuns e capacidade de hosts

CIDR	Máscara decimal	Endereços no bloco	Hosts utilizáveis
/8	255.0.0.0	16.777.216	16.777.214
/16	255.255.0.0	65.536	65.534
/20	255.255.240.0	4.096	4.094
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

Fonte: elaborado pelo SQUAD 1 com base na RFC 4632 [11].

As fórmulas básicas, sendo h o número de bits de host ($h = 32 - n$), são:

$$\text{Endereços no bloco} = 2^{h} \qquad \text{Hosts utilizáveis} = 2^{h} - 2 \tag{9}$$

Subtraem-se 2 porque o primeiro endereço (todos os bits de host em 0) é o **endereço de rede** e o último (todos os bits de host em 1) é o **endereço de broadcast**; nenhum dos dois pode ser atribuído a um host. Exceção: em enlaces ponto a ponto, a RFC 3021 permite usar /31, sem rede nem broadcast.

4.3.1 Como o host descobre a rede: a operação AND

Para saber se o destino está na mesma rede, o host faz um **E lógico bit a bit** (AND) entre o endereço e a máscara. Se o resultado for igual para origem e destino, entrega direto (via ARP); caso contrário, manda para o **gateway padrão**.

Tabela 10: Operação AND entre o endereço 192.168.10.37 e a máscara /26

Item	Binário	Decimal
Endereço	11000000.10101000.00001010.00100101	192.168.10.37
Máscara	11111111.11111111.11111111.11000000	255.255.255.192
AND (rede)	11000000.10101000.00001010.00000000	**192.168.10.0**
Broadcast	11000000.10101000.00001010.00111111	**192.168.10.63**

Fonte: elaborado pelo SQUAD 1.

Analogia para fixar

A máscara funciona como o CEP. Os primeiros dígitos do CEP indicam a região e a rua (parte de rede); o número da casa indica o imóvel (parte de host). O carteiro olha primeiro o CEP para decidir para qual centro de distribuição a carta vai; só no final ele olha o número da casa. O roteador faz a mesma coisa: olha só a parte de rede para decidir o caminho, e só o último roteador se preocupa com o host.

4.4 Divisão em sub-redes (subnetting)

Dividir uma rede em **sub-redes** significa "emprestar" bits da parte de host para a parte de rede. Cada bit emprestado dobra o número de sub-redes e divide pela metade o tamanho de cada uma. Os motivos para fazer isso são: reduzir o tamanho dos domínios de broadcast, separar setores por segurança e organizar o endereçamento.

$$\text{Número de sub-redes} = 2^b \qquad \text{(sendo } b \text{ o número de bits emprestados)} \tag{10}$$

4.4.1 Método do "número mágico"

Um atalho muito usado: o **tamanho do bloco** (ou salto) é $256-$ (valor do octeto da máscara que não é 255 nem 0). Para /26, a máscara é 255.255.255.**192**, então o salto é $256 - 192 = 64$. As sub-redes começam em 0, 64, 128 e 192.

Estudo de Caso 1: Dividindo 192.168.10.0/24 em 4 Sub-redes Iguais

Passo 1 – bits necessários: $2^b \geq 4 \Rightarrow b = 2$. Nova máscara: $/24 + 2 = /26$ (255.255.255.192).
Passo 2 – tamanho do bloco: $2^{32-26} = 64$ endereços, sendo 62 hosts utilizáveis.
Passo 3 – tabela de sub-redes:

Sub-rede	Endereço de rede	Faixa de hosts	Broadcast
1	192.168.10.0/26	.1 a .62	192.168.10.63
2	192.168.10.64/26	.65 a .126	192.168.10.127
3	192.168.10.128/26	.129 a .190	192.168.10.191
4	192.168.10.192/26	.193 a .254	192.168.10.255

4.4.2 VLSM: sub-redes de tamanhos diferentes

Na prática, os setores de uma organização têm tamanhos diferentes. Com **VLSM** (*Variable Length Subnet Mask*), cada sub-rede recebe apenas o tamanho necessário. A regra de ouro é: **alocar primeiro as maiores sub-redes**, para evitar sobreposição e fragmentação do espaço.

Estudo de Caso 2: Planejamento VLSM para uma Empresa

A empresa recebeu o bloco 192.168.1.0/24 e precisa de: Vendas (100 hosts), TI (50 hosts), RH (20 hosts) e um enlace ponto a ponto entre dois roteadores (2 hosts).
Passo 1 – menor máscara que atende cada setor ($2^h - 2 \geq$ necessidade): Vendas: $2^7 - 2 = 126 \Rightarrow$ /25; TI: $2^6 - 2 = 62 \Rightarrow$ /26; RH: $2^5 - 2 = 30 \Rightarrow$ /27; Enlace: $2^2 - 2 = 2 \Rightarrow$ /30.
Passo 2 – alocação em ordem decrescente:

Setor	Máscara	Rede	Hosts utilizáveis	Broadcast
Vendas	/25	192.168.1.0	.1 a .126	.127
TI	/26	192.168.1.128	.129 a .190	.191
RH	/27	192.168.1.192	.193 a .222	.223
Enlace WAN	/30	192.168.1.224	.225 a .226	.227
Livre	–	192.168.1.228	*a 192.168.1.255 (reserva para crescimento)*	

Interpretação: com sub-redes de tamanho fixo /25, só caberiam 2 setores. Com VLSM, os 4 setores foram atendidos e ainda sobraram 28 endereços para expansão.

4.4.3 Agregação de rotas (superredes)

O CIDR também permite o processo inverso: juntar várias redes contíguas num único anúncio. As redes 200.10.0.0/24, 200.10.1.0/24, 200.10.2.0/24 e 200.10.3.0/24 compartilham os primeiros 22 bits e podem ser anunciadas como **200.10.0.0/22**. Isso reduz drasticamente o tamanho das tabelas de roteamento da Internet.

4.5 Endereços especiais, privados e NAT

Tabela 11: Faixas de endereços IPv4 com uso especial

Faixa	Uso
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Endereços **privados** (RFC 1918): uso interno, não roteáveis na Internet
127.0.0.0/8	*Loopback* (o próprio host; 127.0.0.1 = *localhost*)
169.254.0.0/16	Link-local / APIPA: autoconfiguração quando o DHCP falha
100.64.0.0/10	*Carrier-Grade* NAT (CGNAT) usado por operadoras
0.0.0.0	"Este host" / rota padrão (0.0.0.0/0)
255.255.255.255	Broadcast limitado (na rede local)
224.0.0.0/4	Multicast

Fonte: adaptado da RFC 1918 [12] e de Kurose e Ross [1].

4.5.1 NAT (Network Address Translation)

Como os endereços IPv4 públicos acabaram (a IANA distribuiu seus últimos blocos em 2011 e o LACNIC, registro da América Latina, esgotou seu estoque regular em 2020), quase todas as redes domésticas e corporativas usam endereços privados internamente. O **NAT** no roteador de borda troca o IP privado de origem pelo IP público do roteador e guarda a associação numa tabela. No modo mais comum (**NAPT** ou PAT), também troca a porta, permitindo que milhares de dispositivos compartilhem um único IP público.

O NAT resolveu a falta de endereços, mas tem efeitos colaterais: quebra o princípio fim a fim (um host externo não consegue iniciar conexão com um host interno sem configuração de encaminhamento

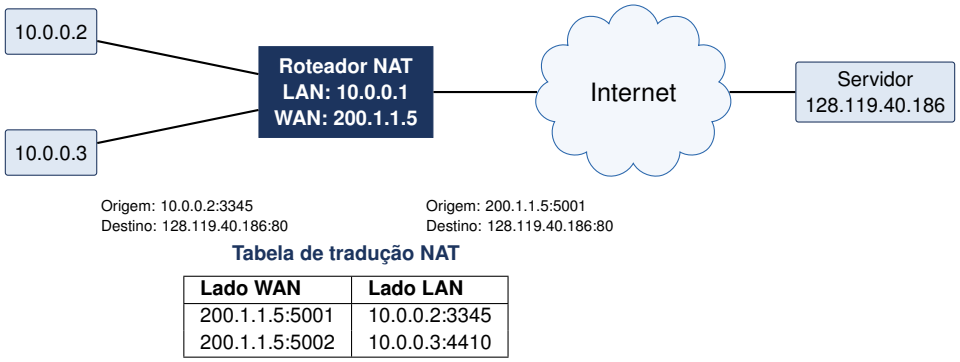


Figura 8: Funcionamento do NAPT: vários hosts privados compartilhando um único IP público

Fonte: adaptado de Kurose e Ross [1].

de portas), dificulta aplicações P2P e VoIP e adiciona estado nos roteadores. Por isso é visto como uma solução "paliativa" até a adoção plena do IPv6.

4.6 IPv6

O **IPv6** (RFC 8200) foi criado principalmente para resolver o esgotamento de endereços, mas também aproveitou para simplificar e melhorar o protocolo.

4.6.1 Endereço de 128 bits

Com 128 bits, há $2^{128} \approx 3{,}4 \times 10^{38}$ endereços, o suficiente para dar trilhões de endereços a cada pessoa do planeta. O endereço é escrito em **oito grupos de 16 bits em hexadecimal**, separados por dois-pontos:

```
2001:0db8:0000:0000:0000:ff00:0042:8329
```

Duas regras de abreviação (RFC 5952):

1. **Zeros à esquerda** de cada grupo podem ser omitidos: `0db8` → `db8`; `0042` → `42`.
2. **Uma única** sequência de grupos inteiramente zerados pode ser substituída por `::`. Só pode aparecer uma vez, senão o endereço fica ambíguo.

Resultado: `2001:db8::ff00:42:8329`.

> **Erro comum**
>
> O endereço `2001:db8:0:0:1:0:0:1` **não** pode ser escrito como `2001:db8::1::1`, porque não daria para saber quantos zeros há em cada "::". A forma correta é `2001:db8::1:0:0:1` (a RFC 5952 recomenda comprimir a sequência mais longa; em caso de empate, a primeira).

4.6.2 Tipos de endereço IPv6

O IPv6 **não tem broadcast**. As funções que dependiam de broadcast (como o ARP) passaram a usar **multicast**. Os principais tipos são:

4.6.3 Autoconfiguração (SLAAC) e EUI-64

Uma das grandes novidades do IPv6 é a **autoconfiguração sem estado** (SLAAC): o host recebe o prefixo /64 por meio de uma mensagem *Router Advertisement* (protocolo NDP, que substitui o ARP) e

Tabela 12: Principais tipos e prefixos de endereços IPv6

Tipo	Prefixo	Descrição
Unicast global	2000::/3	Equivalente ao IP público; roteável na Internet
Link-local	fe80::/10	Válido só no enlace local; toda interface IPv6 tem um, gerado automaticamente
Unique Local (ULA)	fc00::/7	Equivalente aos endereços privados do IPv4
Multicast	ff00::/8	Um para vários (ex.: ff02::1 = todos os nós do enlace)
Anycast	(do espaço unicast)	Um para o mais próximo de um grupo; usado em DNS raiz e CDNs
Loopback	::1/128	O próprio host
Não especificado	::/128	Ausência de endereço (usado na autoconfiguração)

Fonte: adaptado da RFC 4291 e de Tanenbaum, Feamster e Wetherall [2].

gera sozinho os 64 bits finais. Um método clássico é o **EUI-64**, que deriva o identificador do endereço MAC:

1. MAC: `00:1A:2B:3C:4D:5E`. Divide-se ao meio: `001A2B` | `3C4D5E`.
2. Insere-se `FFFE` no meio: `001A2B`**`FFFE`**`3C4D5E`.
3. Inverte-se o 7º bit do primeiro byte: `00` = `00000000` → `00000010` = `02`.
4. Identificador: `021a:2bff:fe3c:4d5e`. Com o prefixo `2001:db8:1:1::/64`, o endereço fica `2001:db8:1:1:21a:2bff:fe3c:4d5e`.

Como o EUI-64 expõe o MAC e permite rastrear o dispositivo, os sistemas operacionais modernos usam por padrão identificadores aleatórios e temporários (extensões de privacidade, RFC 8981).

4.6.4 Cabeçalho simplificado

O cabeçalho IPv6 tem **tamanho fixo de 40 bytes** e apenas 8 campos. Foram removidos o checksum (as camadas de enlace e transporte já verificam erros), os campos de fragmentação (no IPv6 **roteadores não fragmentam**; só a origem, após descobrir a MTU do caminho) e as opções (substituídas por cabeçalhos de extensão encadeados pelo campo “próximo cabeçalho”). Tudo isso acelera o processamento nos roteadores.

Tabela 13: Comparação entre IPv4 e IPv6

Característica	IPv4	IPv6
Tamanho do endereço	32 bits	128 bits
Notação	Decimal pontuada	Hexadecimal com dois-pontos
Cabeçalho	20 a 60 bytes, variável	40 bytes, fixo
Checksum no cabeçalho	Sim	Não
Fragmentação	Origem e roteadores	Apenas origem
Broadcast	Sim	Não (usa multicast)
Resolução de endereço físico	ARP	NDP (ICMPv6)
Autoconfiguração	DHCP	SLAAC e/ou DHCPv6
IPsec	Opcional	Suporte previsto na especificação
Necessidade de NAT	Muito comum	Desnecessário

Fonte: adaptado de Kurose e Ross [1] e da RFC 8200 [13].

4.6.5 Mecanismos de transição

Como não é possível "desligar" o IPv4 de uma vez, a Internet vive um longo período de convivência:

- **Pilha dupla (*dual stack*)**: os equipamentos rodam IPv4 e IPv6 ao mesmo tempo. É a abordagem mais usada.
- **Tunelamento**: pacotes IPv6 são encapsulados dentro de pacotes IPv4 para atravessar trechos que só entendem IPv4 (6in4, 6to4, 6rd).
- **Tradução**: NAT64 e DNS64 permitem que hosts apenas IPv6 acessem serviços apenas IPv4; o 464XLAT é muito usado por operadoras móveis.

Estudo de Caso 3: Identificando a Rede de um Host com Máscara Não Trivial

Um host tem o endereço **172.16.45.130/20**. Determine a rede, o broadcast, a faixa de hosts e a quantidade de hosts utilizáveis.
Passo 1 – máscara: /20 = 255.255.**240**.0. O octeto "interessante" é o terceiro.
Passo 2 – salto: $256 - 240 = 16$. As redes no terceiro octeto começam em 0, 16, 32, 48…
Passo 3 – localizar: o terceiro octeto do host é 45, que está entre 32 e 47. Logo:

- Rede: **172.16.32.0/20**
- Broadcast: **172.16.47.255**
- Faixa de hosts: 172.16.32.1 a 172.16.47.254
- Hosts utilizáveis: $2^{12} - 2 =$ **4.094**

Conferência em binário (3º octeto): $45 = 0010\,1101$; AND com $240 = 1111\,0000$ resulta em $0010\,0000 = 32$. ✓

Questão comentada (estilo ENADE)

Um administrador precisa dividir a rede 10.20.0.0/16 de forma que cada sub-rede comporte pelo menos 500 hosts, maximizando o número de sub-redes. Qual máscara deve ser usada e quantas sub-redes serão obtidas?
(A) /23 e 128 sub-redes **(B)** /24 e 256 sub-redes **(C)** /22 e 64 sub-redes **(D)** /23 e 64 sub-redes **(E)** /25 e 512 sub-redes
Resposta: (A). Para 500 hosts, é preciso $2^h - 2 \geq 500$, o que dá $h = 9$ ($2^9 - 2 = 510$). A máscara fica $32 - 9 = /23$. Os bits emprestados são $23 - 16 = 7$, gerando $2^7 = 128$ sub-redes. A alternativa (B) comporta apenas 254 hosts por sub-rede; a (C) atende aos hosts, mas não maximiza o número de sub-redes.

5 Protocolos de Roteamento e Camada de Enlace

Este capítulo junta duas camadas que trabalham lado a lado. A **camada de rede** decide o caminho que o pacote vai seguir entre redes diferentes; a **camada de enlace** cuida de cada "pulo" desse caminho, levando o quadro de um nó até o nó vizinho. Uma boa forma de lembrar: o roteamento é o planejamento da viagem, e o enlace é cada trecho percorrido.

5.1 Encaminhamento e roteamento

Kurose e Ross [1] fazem uma distinção que costuma cair em prova:

- **Encaminhamento (*forwarding*)**: ação **local** do roteador, que move o pacote da porta de entrada

para a porta de saída correta. Acontece em nanossegundos, normalmente em hardware. Faz parte do **plano de dados**.

- **Roteamento (*routing*)**: processo **global** que calcula os caminhos fim a fim e preenche as tabelas. Acontece em segundos, em software. Faz parte do **plano de controle**.

5.1.1 A tabela de roteamento e o casamento do prefixo mais longo

A tabela de roteamento associa prefixos de destino a interfaces de saída (ou próximos saltos). Como um mesmo endereço pode casar com vários prefixos, o roteador aplica a regra do **casamento do prefixo mais longo** (*longest prefix match*): vence a entrada mais específica, ou seja, a de maior máscara.

Tabela 14: Exemplo de tabela de roteamento

Prefixo de destino	Interface de saída	Observação
200.23.16.0/21	0	Cobre 200.23.16.0 a 200.23.23.255
200.23.24.0/24	1	Cobre 200.23.24.0 a 200.23.24.255
200.23.24.0/21	2	Cobre 200.23.24.0 a 200.23.31.255
0.0.0.0/0	3	Rota padrão (casa com tudo)

Fonte: adaptado de Kurose e Ross [1].

- Destino **200.23.22.161**: casa com a primeira entrada (/21) e com a rota padrão (/0). Vence a /21 $\rightarrow$ interface 0.
- Destino **200.23.24.170**: casa com a /24, com a /21 da terceira linha e com a /0. Vence a /24, que é a mais longa $\rightarrow$ interface 1.
- Destino **8.8.8.8**: só casa com a rota padrão $\rightarrow$ interface 3.

5.2 Algoritmos de roteamento

Para o algoritmo de roteamento, a rede é um **grafo** $G = (N, E)$: os nós são roteadores e as arestas são enlaces, cada uma com um **custo** (que pode representar distância, atraso, inverso da banda ou custo financeiro). O objetivo é encontrar o **caminho de menor custo** entre os nós. Há duas grandes famílias:

Tabela 15: Estado de enlace $\times$ vetor de distâncias

Critério	Estado de enlace (LS)	Vetor de distâncias (DV)
Informação usada	Global: cada roteador conhece a topologia inteira	Descentralizada: cada roteador conhece só os vizinhos
Algoritmo	Dijkstra	Bellman-Ford distribuído
O que é trocado	Estado dos seus enlaces, enviado a **todos** (inundação)	Sua tabela de distâncias, enviada só aos **vizinhos**
Convergência	Rápida	Pode ser lenta (contagem ao infinito)
Robustez	Erro de um nó afeta só o cálculo dele	Erro de um nó se propaga pela rede
Exemplos	OSPF, IS-IS	RIP, (base do) BGP

Fonte: adaptado de Kurose e Ross [1] e Tanenbaum, Feamster e Wetherall [2].

5.2.1 Algoritmo de Dijkstra (estado de enlace)

O algoritmo de Dijkstra calcula, a partir de um nó de origem u, o menor custo até todos os outros nós. Ele mantém um conjunto N' de nós cujo menor caminho já é definitivo. A cada iteração, adiciona a N' o

nó w ainda não incluído com menor distância $D(w)$ e atualiza os vizinhos:

$$D(v) \leftarrow \min\big(D(v),\ D(w) + c(w,v)\big) \tag{11}$$

Com n nós, a implementação simples tem complexidade $O(n^2)$; com fila de prioridade (heap), cai para $O(|E| \log n)$.

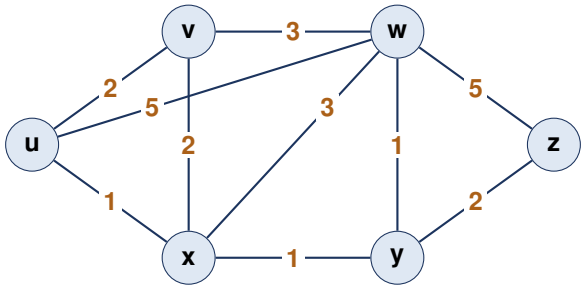


Figura 9: Grafo de exemplo: os números indicam o custo de cada enlace

Fonte: adaptado de Kurose e Ross [1].

Estudo de Caso: Executando Dijkstra a partir do Nó u

Cada célula mostra *distância, predecessor*. O traço indica que o nó já está em N' (valor definitivo).

Passo	N'	$D(v), p$	$D(w), p$	$D(x), p$	$D(y), p$	$D(z), p$
0	u	2, u	5, u	**1, u**	∞	∞
1	ux	**2, u**	4, x	–	2, x	∞
2	uxv	–	4, x	–	**2, x**	∞
3	uxvy	–	**3, y**	–	–	4, y
4	uxvyw	–	–	–	–	**4, y**
5	uxvywz	–	–	–	–	–

Leitura dos passos: no passo 0, o menor valor é $D(x) = 1$, e x entra em N'. Por x, w passa a custar $1 + 3 = 4$ e y passa a custar $1 + 1 = 2$. No passo 1, há empate entre v e y (ambos 2); escolhemos v, que não melhora ninguém. No passo 2 entra y: w cai para $2 + 1 = 3$ e z passa a $2 + 2 = 4$. Depois entram w e z.
Resultado – tabela de encaminhamento de u: para chegar a v, sai pelo enlace (u, v); para x, y, w e z, sai pelo enlace (u, x). Os custos finais são $v = 2$, $x = 1$, $y = 2$, $w = 3$ e $z = 4$.

5.2.2 Vetor de distâncias e a equação de Bellman-Ford

No roteamento por vetor de distâncias, cada nó x estima seu custo até cada destino y usando apenas o que os vizinhos informam. A base é a **equação de Bellman-Ford**:

$$D_x(y) = \min_{v \in \text{vizinhos}(x)} \big\{ c(x,v) + D_v(y) \big\} \tag{12}$$

Ou seja: “meu custo até y é o menor valor entre (custo até o vizinho v) + (o que v diz que gasta até y)”. O processo é **iterativo** (repete até nada mudar), **assíncrono** e **distribuído**.

Problema da contagem ao infinito. Boas notícias se espalham rápido, mas más notícias se espalham devagar. Se o enlace entre y e x cai, y pode achar que ainda chega a x passando por z, sem saber que o caminho de z passava justamente por y. Os dois ficam trocando valores cada vez maiores até atingir o “infinito” (no RIP, 16 saltos). Técnicas de mitigação: **horizonte dividido** (*split horizon*), **envenenamento reverso** (*poisoned reverse*) e *hold-down timers*.

5.3 Roteamento na Internet: sistemas autônomos

A Internet é grande demais para um único algoritmo rodar sobre todos os roteadores. Por isso ela é dividida em **Sistemas Autônomos (AS)**: conjuntos de redes sob uma mesma administração (uma operadora, uma universidade, um grande provedor de conteúdo). Cada AS é identificado por um número (ASN). Isso gera dois níveis de roteamento:

- **Intra-AS (IGP – *Interior Gateway Protocol*)**: dentro de um AS. O foco é **desempenho** (menor custo). Exemplos: RIP, OSPF, IS-IS, EIGRP.
- **Inter-AS (EGP – *Exterior Gateway Protocol*)**: entre ASes. O foco é **política** (quem pode passar tráfego por quem, acordos comerciais). O único usado na prática é o **BGP**.

5.3.1 Principais protocolos

Tabela 16: Comparação dos principais protocolos de roteamento

Protocolo	Tipo	Métrica	Características
RIP (v2)	IGP, vetor de distâncias	Número de saltos (máx. 15)	Simples; atualizações a cada 30 s via UDP 520; só para redes pequenas
OSPF (v2/v3)	IGP, estado de enlace	Custo (baseado na banda)	Usa Dijkstra; divide o AS em áreas (área 0 = backbone); convergência rápida; roda direto sobre IP (protocolo 89)
IS-IS	IGP, estado de enlace	Custo configurável	Muito usado em grandes operadoras; independente do IP
EIGRP	IGP, vetor de distâncias avançado	Banda e atraso (composta)	Originalmente da Cisco; algoritmo DUAL evita laços
BGP-4	EGP, vetor de caminhos	Atributos e políticas	Anuncia o caminho completo de ASes (AS-PATH), o que evita laços; roda sobre TCP 179

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2], RFC 2328 e RFC 4271.

Como o BGP escolhe a rota? Quando há várias rotas para o mesmo prefixo, o BGP aplica, em ordem: (1) a maior *local preference* (política definida pelo administrador); (2) o menor AS-PATH; (3) o próximo salto mais próximo (*hot potato*, "batata quente": livrar-se do pacote o quanto antes); (4) critérios de desempate adicionais. Um detalhe importante: o BGP não busca necessariamente o caminho mais rápido, e sim o caminho permitido pelos acordos comerciais entre as empresas.

5.3.2 ICMP

O **ICMP** (*Internet Control Message Protocol*) é o "protocolo de mensagens de erro" da camada de rede. É usado pelo `ping` (mensagens *Echo Request*/*Echo Reply*, tipos 8 e 0) e pelo `traceroute` (mensagem *Time Exceeded*, tipo 11, gerada quando o TTL chega a zero). Também avisa quando um destino é inalcançável (tipo 3).

5.4 Camada de enlace

A camada de enlace transfere quadros entre nós **adjacentes**. Ela é implementada principalmente no hardware da placa de rede (NIC). Seus serviços principais são [1, 3]:

1. **Enquadramento**: encapsular o datagrama num quadro com cabeçalho e trailer.
2. **Acesso ao enlace**: regras para decidir quem transmite quando o meio é compartilhado (protocolo MAC).

3. **Entrega confiável entre vizinhos**: comum em enlaces com muitos erros (como o Wi-Fi); em enlaces com fibra ou cobre de boa qualidade, normalmente não é usada.
4. **Detecção e correção de erros**: bits podem ser invertidos por ruído ou atenuação.

5.4.1 Detecção de erros: paridade, checksum e CRC

- **Paridade simples**: acrescenta 1 bit para que o total de bits "1" seja par (ou ímpar). Detecta qualquer número **ímpar** de erros, mas falha se dois bits forem invertidos.
- **Paridade bidimensional**: organiza os bits em matriz e calcula paridade por linha e por coluna. Consegue **detectar e corrigir** um erro de bit único (o erro fica no cruzamento da linha e da coluna com paridade errada).
- **Checksum**: soma dos dados em palavras de 16 bits, em complemento de 1. Simples de calcular em software; usado no IP, TCP e UDP.
- **CRC (*Cyclic Redundancy Check*)**: trata os bits como coeficientes de um polinômio e usa divisão módulo 2 por um **gerador** G de $r + 1$ bits. É muito mais robusto e é implementado em hardware. O Ethernet usa o CRC-32.

Estudo de Caso: Calculando o CRC

Dados $D = 101110$ e gerador $G = 1001$ (portanto $r = 3$). Quais bits de CRC (R) devem ser enviados?
Passo 1: acrescentar $r = 3$ zeros aos dados: $101110\,000$.
Passo 2: dividir por G usando XOR (divisão módulo 2, sem "vai um"):

```
101110000      <- dados + 3 zeros
1001           <- XOR com G
---------
001010000
  1001         <- XOR com G (alinhado ao primeiro 1)
---------
000011000
    1001
---------
000001010
     1001
---------
000000011      <- resto R = 011
```

Passo 3: o quadro transmitido é D seguido de R: **101110 011**.
Verificação no receptor: ao dividir 101110011 por 1001, o resto deve ser **zero**. Se não for, houve erro, e o quadro é descartado. O CRC com r bits detecta todas as rajadas de erro com até r bits.

5.4.2 Protocolos de acesso múltiplo

Quando vários nós compartilham o mesmo meio (um barramento, um canal de rádio), é preciso evitar que todos falem ao mesmo tempo. As três famílias são:

- **Divisão de canal**: TDMA (fatias de tempo), FDMA (faixas de frequência), CDMA (códigos). Justo, mas desperdiça recursos quando poucos nós transmitem.
- **Acesso aleatório**: cada nó transmite quando quer e lida com colisões. Exemplos: ALOHA, CSMA, **CSMA/CD** (Ethernet clássico) e **CSMA/CA** (Wi-Fi).

- **Revezamento**: *polling* (um nó mestre chama os outros) ou passagem de *token*.

No **CSMA/CD** (*Carrier Sense Multiple Access with Collision Detection*), o nó escuta o meio antes de transmitir; se estiver livre, transmite, e continua escutando. Ao detectar colisão, interrompe, envia um sinal de *jam* e espera um tempo aleatório antes de tentar de novo. Esse tempo é dado pelo **recuo exponencial binário**: após a n-ésima colisão, escolhe-se K ao acaso em $\{0, 1, \ldots, 2^{\min(n,10)} - 1\}$ e espera-se $K \times 512$ tempos de bit. Com switches e enlaces *full-duplex*, o CSMA/CD deixou de ser necessário, mas continua no padrão por compatibilidade.

5.4.3 Endereço MAC e o quadro Ethernet

O **endereço MAC** tem 48 bits, escritos em hexadecimal (ex.: `3C:22:FB:9A:10:7E`). Os primeiros 24 bits formam o **OUI**, que identifica o fabricante (atribuído pelo IEEE); os últimos 24 são definidos pelo fabricante. Diferente do IP, o MAC é "plano" (não hierárquico): funciona como um CPF, enquanto o IP funciona como um endereço postal.

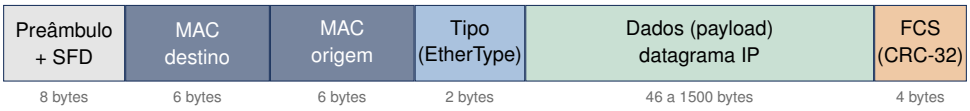


Figura 10: Formato do quadro Ethernet II (IEEE 802.3)

Fonte: adaptado de Kurose e Ross [1] e Forouzan [3].

O tamanho mínimo de dados (46 bytes, totalizando 64 bytes de quadro) existe para garantir que, no Ethernet compartilhado, a transmissão dure o suficiente para que uma colisão seja detectada antes do fim do envio.

5.4.4 ARP: de IP para MAC

Para mandar um datagrama a um host (ou ao gateway) na mesma rede, é preciso saber seu MAC. O **ARP** (*Address Resolution Protocol*) resolve isso:

1. O host A consulta sua **tabela ARP** (cache). Se não encontrar o IP de destino, envia um **ARP Request em broadcast**: "Quem tem o IP 192.168.1.1? Responda para 192.168.1.10".
2. Só o dono daquele IP responde com um **ARP Reply em unicast**, contendo seu MAC.
3. A guarda a associação IP–MAC no cache por alguns minutos.

> **Segurança**
>
> O ARP não tem autenticação: qualquer host pode responder dizendo "o IP do gateway sou eu". Esse ataque, chamado de **ARP spoofing** (ou envenenamento de ARP), permite interceptar o tráfego da rede local (*man-in-the-middle*). Defesas incluem *Dynamic ARP Inspection* nos switches e o uso de criptografia fim a fim (Capítulo 8).

5.4.5 VLANs e Spanning Tree

Uma **VLAN** (IEEE 802.1Q) divide logicamente um switch em várias redes: cada VLAN é um domínio de broadcast separado, e entre switches os quadros levam uma etiqueta (*tag*) de 4 bytes com o número da VLAN (12 bits). Para VLANs diferentes conversarem, é preciso um roteador ou switch de camada 3. Já o **STP** (IEEE 802.1D) evita laços em redes com enlaces redundantes entre switches: como quadros Ethernet não têm TTL, um broadcast circularia para sempre (*broadcast storm*). O STP elege um switch raiz e bloqueia portas até a topologia virar uma árvore, reativando-as se um enlace falhar.

Questão comentada (estilo ENADE)

Um roteador possui as seguintes rotas: 10.0.0.0/8 → R1; 10.1.0.0/16 → R2; 10.1.2.0/24 → R3; 0.0.0.0/0 → R4. Os pacotes destinados a 10.1.2.9, 10.1.5.7 e 172.16.0.1 serão enviados, respectivamente, para:
(A) R1, R1 e R4 **(B)** R3, R2 e R4 **(C)** R3, R2 e R1 **(D)** R2, R2 e R4 **(E)** R3, R1 e R4
Resposta: (B). Pela regra do prefixo mais longo: 10.1.2.9 casa com /8, /16 e /24, e vence a /24 (R3); 10.1.5.7 casa com /8 e /16, e vence a /16 (R2); 172.16.0.1 só casa com a rota padrão (R4). A alternativa (A) é o erro de escolher a primeira rota que casa, e não a mais específica.

6 Camada de Transporte: Protocolos TCP e UDP

A camada de rede entrega pacotes entre **hosts**. Mas, dentro de cada host, há vários programas usando a rede ao mesmo tempo: o navegador, o cliente de e-mail, o Spotify, o Discord. A camada de transporte estende a entrega de host a host para uma comunicação **lógica entre processos** [1]. Ela existe apenas nos sistemas finais: roteadores não olham (em princípio) para os cabeçalhos de transporte.

6.1 Serviços da camada de transporte

O IP é um serviço de **melhor esforço**: pacotes podem ser perdidos, duplicados, corrompidos ou chegar fora de ordem. Sobre essa base pouco confiável, a camada de transporte pode oferecer:

- **Multiplexação e demultiplexação** (TCP e UDP): juntar dados de vários processos na origem e entregar ao processo certo no destino, usando **portas**.
- **Verificação de integridade** (TCP e UDP): checksum.
- **Entrega confiável e ordenada** (só TCP).
- **Controle de fluxo** (só TCP): não afogar o receptor.
- **Controle de congestionamento** (só TCP): não afogar a rede.

Nenhum dos dois protocolos oferece garantia de atraso máximo ou de vazão mínima; essas garantias simplesmente não existem na Internet “pura”.

6.2 Portas e sockets

Uma **porta** é um número de 16 bits (0 a 65.535) que identifica um processo dentro do host. A IANA divide as portas em três faixas:

- **0 a 1.023 – portas bem conhecidas**: reservadas para serviços padronizados.
- **1.024 a 49.151 – portas registradas**: usadas por aplicações específicas (ex.: 3306 MySQL, 3389 RDP, 5432 PostgreSQL).
- **49.152 a 65.535 – portas dinâmicas ou efêmeras**: escolhidas pelo sistema operacional para o lado cliente de uma conexão.

Um **socket** é a interface entre a aplicação e a camada de transporte, ou seja, a “porta de casa” pela qual o processo envia e recebe mensagens. A diferença entre os dois protocolos aparece na demultiplexação:

- Um socket **UDP** é identificado pela dupla *(IP de destino, porta de destino)*. Dois datagramas de origens diferentes, mas para a mesma porta, chegam ao mesmo socket.

Tabela 17: Portas bem conhecidas mais cobradas

Porta	Serviço	Transp.	Porta	Serviço	Transp.
20/21	FTP (dados/controle)	TCP	110	POP3	TCP
22	SSH / SFTP	TCP	123	NTP	UDP
23	Telnet	TCP	143	IMAP	TCP
25	SMTP	TCP	161/162	SNMP	UDP
53	DNS	UDP e TCP	443	HTTPS (e HTTP/3 via UDP)	TCP/UDP
67/68	DHCP (servidor/cliente)	UDP	587	SMTP (submissão)	TCP
80	HTTP	TCP	993	IMAPS	TCP

Fonte: adaptado dos registros da IANA e de Comer [4].

- Um socket **TCP** é identificado pela quádrupla *(IP de origem, porta de origem, IP de destino, porta de destino)*. Um servidor web na porta 443 cria um socket separado para cada cliente conectado.

6.3 UDP: User Datagram Protocol

O **UDP** (RFC 768 [23]) é o protocolo de transporte mais simples possível: acrescenta ao IP apenas multiplexação por portas e um checksum. Não há conexão, confirmação, retransmissão, ordenação, controle de fluxo nem de congestionamento.

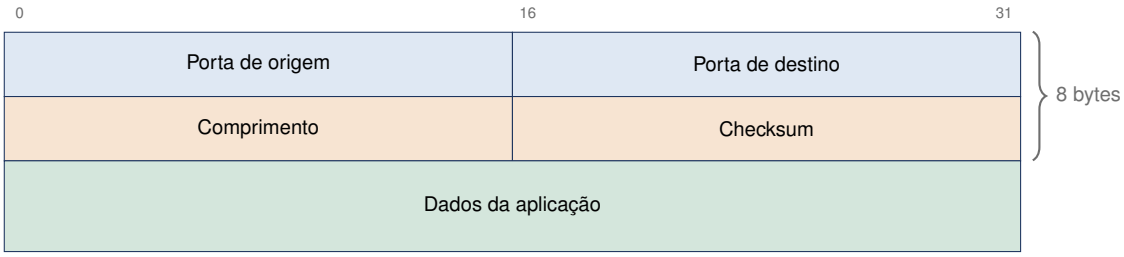


Figura 11: Formato do segmento UDP

Fonte: adaptado da RFC 768 e de Kurose e Ross [1].

Por que alguém usaria um protocolo que não garante nada?

- **Menor atraso**: não há apresentação inicial (*handshake*); o primeiro pacote já leva dados.
- **Sem estado de conexão**: o servidor aguenta muito mais clientes simultâneos (importante para DNS).
- **Cabeçalho pequeno**: 8 bytes contra 20 do TCP.
- **Controle total pela aplicação**: em chamadas de vídeo e jogos online, um pacote atrasado é inútil; é melhor perder e seguir em frente do que esperar a retransmissão.

Exemplos de uso: DNS, DHCP, SNMP, NTP, VoIP e videoconferência (RTP), jogos online e o **QUIC**, protocolo sobre UDP que implementa confiabilidade e criptografia no espaço do usuário e é a base do HTTP/3 (Capítulo 7).

6.4 Princípios de transferência confiável

Qualquer protocolo confiável combina cinco mecanismos [1]: **checksum** (detectar bits corrompidos), **reconhecimento**/**ACK** (confirmar o recebimento), **temporizador** (retransmitir se o ACK não vier), **número de sequência** (detectar duplicatas e reordenar) e **janela** (enviar vários pacotes sem esperar cada ACK).

6.4.1 Por que não enviar um pacote de cada vez? (Pare e Espere)

No protocolo **pare e espere** (*stop-and-wait*), o transmissor envia um pacote e fica parado até receber o ACK. A utilização do enlace é:

$$U = \frac{L/R}{RTT + L/R} \tag{13}$$

Num enlace de 1 Gbps com RTT de 30 ms e pacotes de 1.000 bytes: $L/R = 8000/10^9 = 8\ \mu s$, e $U = 0{,}008/30{,}008 \approx 0{,}00027$. O enlace fica ocioso **99,97%** do tempo! A solução é o **paralelismo**: enviar uma **janela** de N pacotes antes de parar. Com N pacotes, a utilização passa a ser N vezes maior (até o limite de 100%). As duas estratégias clássicas são:

- **Go-Back-N**: o receptor só aceita pacotes em ordem e envia ACK cumulativo. Se um pacote se perde, o transmissor reenvia ele e **todos** os seguintes.
- **Repetição seletiva**: o receptor guarda pacotes fora de ordem e confirma cada um individualmente; o transmissor reenvia **apenas** o que foi perdido.

O TCP usa um híbrido: ACKs cumulativos (como no Go-Back-N), mas normalmente retransmite só o segmento perdido e o receptor guarda os que chegaram fora de ordem (como na repetição seletiva).

6.5 TCP: Transmission Control Protocol

O **TCP** (RFC 9293, que consolidou a especificação original de 1981) oferece um serviço **orientado a conexão**, **confiável**, **ordenado**, **full-duplex** e baseado em **fluxo de bytes** (a aplicação escreve bytes, não mensagens; o TCP decide como dividir em segmentos).

6.5.1 Estrutura do segmento TCP

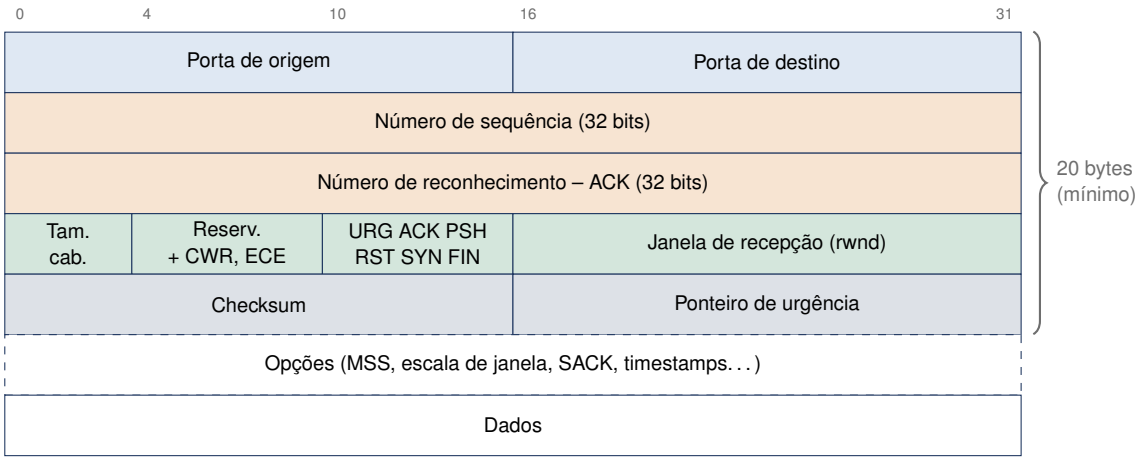


Figura 12: Formato do segmento TCP

Fonte: adaptado da RFC 9293 [14] e de Kurose e Ross [1].

Dois campos merecem atenção especial:

- O **número de sequência** conta **bytes**, e não segmentos. Ele indica a posição do primeiro byte de dados do segmento no fluxo. O valor inicial (ISN) é escolhido de forma pseudoaleatória, por segurança.
- O **número de reconhecimento** é o número do **próximo byte esperado**. Se o receptor já recebeu corretamente os bytes 0 a 999, envia ACK = 1000. Por isso o ACK do TCP é **cumulativo**.

6.5.2 Estabelecimento de conexão: apresentação em três vias

Antes de trocar dados, cliente e servidor sincronizam seus números de sequência iniciais com o famoso **three-way handshake**:

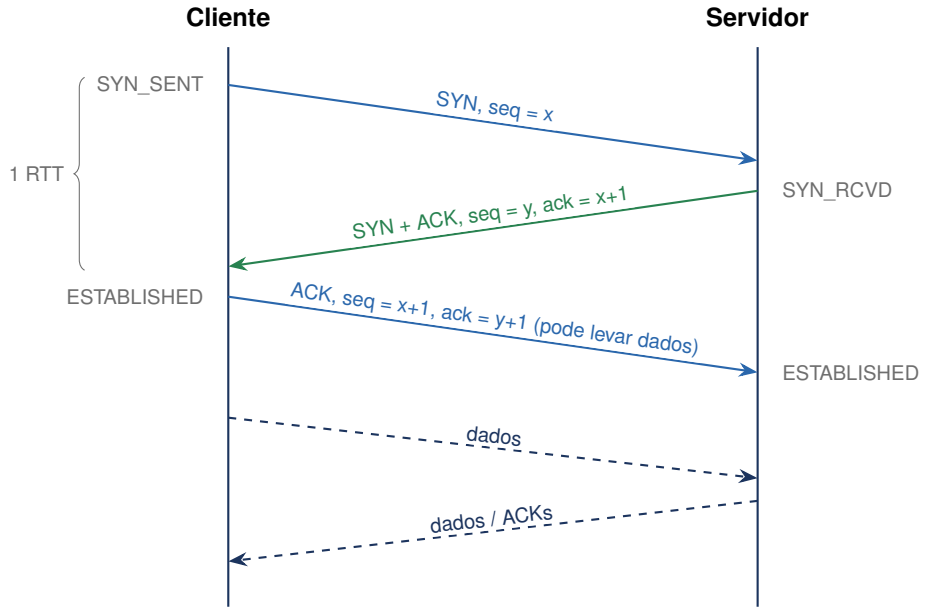


Figura 13: Diagrama de sequência da apresentação em três vias do TCP

Fonte: adaptado de Kurose e Ross [1] e da RFC 9293 [14].

Por que três vias e não duas? Com duas mensagens, o servidor não teria como saber se o cliente recebeu o seu número de sequência inicial, e um SYN antigo e atrasado na rede poderia abrir uma conexão "fantasma". A terceira mensagem confirma que os dois lados estão sincronizados.

> **Ataque SYN flood**
>
> Um atacante pode enviar milhares de SYNs com IPs de origem falsos e nunca responder ao SYN+ACK. O servidor fica com muitas conexões "meio abertas" ocupando memória, até não conseguir aceitar clientes legítimos. Uma defesa elegante são os **SYN cookies**: o servidor não guarda estado até receber o ACK final, codificando as informações necessárias no próprio número de sequência que ele envia.

6.5.3 Encerramento de conexão

Como a conexão é *full-duplex*, cada sentido é fechado de forma independente, normalmente em **quatro mensagens**: FIN (A $\rightarrow$ B), ACK (B $\rightarrow$ A), FIN (B $\rightarrow$ A) e ACK (A $\rightarrow$ B). Quem fecha primeiro fica no estado `TIME_WAIT` por um tempo (2 $\times$ MSL) para garantir que o último ACK chegou e que segmentos antigos não contaminem uma nova conexão. Uma conexão também pode ser abortada imediatamente com um segmento **RST**.

6.5.4 Estimativa do RTT e temporizador de retransmissão

O TCP precisa decidir quanto tempo esperar antes de retransmitir. Se o tempo for curto demais, retransmite à toa; se for longo demais, demora a se recuperar de perdas. A solução é medir o RTT

continuamente (*SampleRTT*) e calcular médias móveis ponderadas exponencialmente (RFC 6298 [22]):

$$\text{DevRTT} = (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}| \quad (\beta = 0{,}25) \tag{14}$$

$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT} \quad (\alpha = 0{,}125) \tag{15}$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\cdot\text{DevRTT} \tag{16}$$

Estudo de Caso: Calculando o Timeout do TCP

Situação atual: EstimatedRTT = 100 ms e DevRTT = 10 ms. Chega uma nova medição: SampleRTT = 120 ms.
Passo 1 – atualizar DevRTT (com o EstimatedRTT ainda antigo, como manda a RFC 6298): DevRTT $= 0{,}75 \times 10 + 0{,}25 \times |120 - 100| = 7{,}5 + 5 = 12{,}5$ ms.
Passo 2 – atualizar EstimatedRTT: EstimatedRTT $= 0{,}875 \times 100 + 0{,}125 \times 120 = 87{,}5 + 15 = 102{,}5$ ms.
Passo 3 – novo timeout: TimeoutInterval $= 102{,}5 + 4 \times 12{,}5 =$ **152,5 ms**.
Interpretação: um único RTT maior que o normal quase não mexe na média (subiu só 2,5 ms), mas a margem de segurança ($4\times$ DevRTT) cresce, porque a rede mostrou mais variação. Além disso, sempre que ocorre um timeout, o TCP **dobra** o intervalo para a próxima tentativa (recuo exponencial).

6.5.5 Retransmissão rápida

Esperar o timeout pode ser lento. Se o receptor recebe um segmento fora de ordem, ele reenvia o ACK do último byte em ordem (ACK duplicado). Quando o transmissor recebe **três ACKs duplicados**, ele conclui que o segmento seguinte se perdeu e o retransmite imediatamente, sem esperar o temporizador. Isso é a **retransmissão rápida** (*fast retransmit*).

6.5.6 Controle de fluxo

O **controle de fluxo** impede que o transmissor encha o buffer do receptor. O receptor anuncia, em todo segmento, quanto espaço livre tem no campo **janela de recepção** (*rwnd*). O transmissor garante que:

$$\text{ÚltimoByteEnviado} - \text{ÚltimoByteConfirmado} \leq \text{rwnd} \tag{17}$$

É um problema de **velocidade entre dois computadores**: um servidor potente mandando dados para um celular antigo, por exemplo.

6.5.7 Controle de congestionamento

Já o **controle de congestionamento** protege a **rede**: se todo mundo transmitir o máximo possível, os buffers dos roteadores enchem, pacotes são descartados, as retransmissões pioram ainda mais a situação e a vazão útil despenca (o chamado “colapso por congestionamento”, observado na Internet em 1986). O TCP controla sua taxa com uma segunda janela, a **janela de congestionamento** (*cwnd*), e envia no máximo $\min(\text{cwnd}, \text{rwnd})$ bytes não confirmados. A vazão aproximada é:

$$\text{Vazão} \approx \frac{\text{cwnd}}{RTT} \tag{18}$$

O TCP clássico (Reno) tem três fases:

1. **Partida lenta (*slow start*)**: começa com cwnd = 1 MSS e **dobra** a cada RTT (crescimento exponencial) até atingir o limiar *ssthresh*. O nome engana: é “lenta” só no começo.

2. **Prevenção de congestionamento**: acima do ssthresh, cresce **1 MSS por RTT** (crescimento linear, “aditivo”).
3. **Reação à perda**:
   - **3 ACKs duplicados** (perda leve, a rede ainda entrega segmentos): ssthresh = cwnd/2 e cwnd cai pela metade (**recuperação rápida**). É o “decréscimo multiplicativo”.
   - **Timeout** (perda grave): ssthresh = cwnd/2 e cwnd volta a **1 MSS**, reiniciando a partida lenta.

Esse comportamento é chamado de **AIMD** (*Additive Increase, Multiplicative Decrease*) e produz o famoso gráfico “dente de serra”.

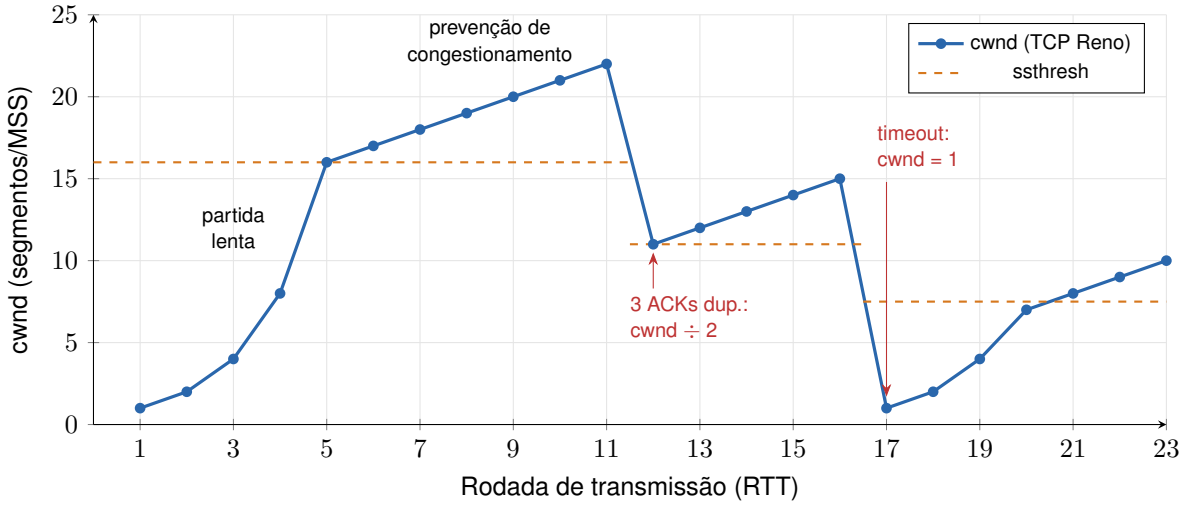


Figura 14: Evolução da janela de congestionamento no TCP Reno: partida lenta, AIMD, perda por ACKs duplicados e por timeout

Fonte: elaborado pelo SQUAD 1 com base em Kurose e Ross [1] e na RFC 5681.

Leitura do gráfico: com ssthresh inicial de 16, a janela dobra até 16 (rodadas 1 a 5) e depois cresce de 1 em 1. Na rodada 11, com cwnd = 22, chegam 3 ACKs duplicados: ssthresh e cwnd passam a 11. Na rodada 16, com cwnd = 15, ocorre um timeout: ssthresh vai para 7 (metade de 15, arredondada para baixo) e cwnd volta a 1. A partida lenta recomeça e, ao atingir o ssthresh (rodada 20), a janela volta a crescer linearmente.

> **Analogia para fixar**
>
> O controle de congestionamento é como dirigir numa estrada desconhecida à noite. Você começa devagar e vai acelerando cada vez mais rápido enquanto a estrada parece livre (partida lenta). Quando se aproxima da velocidade em que já teve problemas antes (ssthresh), passa a acelerar com cuidado (crescimento linear). Se aparece um sinal de alerta (ACKs duplicados), você reduz pela metade; se quase bate (timeout), freia quase até parar e recomeça com cautela.

Hoje, a maioria dos sistemas usa variantes mais modernas. O **CUBIC** (padrão no Linux, Windows e macOS) cresce a janela segundo uma função cúbica do tempo desde a última perda, aproveitando melhor enlaces de alta velocidade e longa distância. O **BBR**, do Google, tenta estimar a banda do gargalo e o RTT mínimo em vez de esperar perdas para reagir.

Justiça. Se K conexões TCP dividem um gargalo de capacidade R, o AIMD faz cada uma tender a R/K, sem nenhuma coordenação central: um bom exemplo de compartilhamento de recursos (DCN, Art. 5º, §1º, X).

6.6 TCP × UDP: quadro-resumo

Tabela 18: Comparação entre TCP e UDP

Característica	TCP	UDP
Conexão	Orientado a conexão (handshake)	Sem conexão
Confiabilidade	Garante entrega, sem erros e sem duplicatas	Não garante
Ordem	Entrega em ordem	Sem garantia de ordem
Controle de fluxo	Sim (rwnd)	Não
Controle de congestionamento	Sim (cwnd, AIMD)	Não
Unidade	Fluxo de bytes	Mensagens (datagramas)
Cabeçalho	20 a 60 bytes	8 bytes
Broadcast/multicast	Não	Sim
Usos típicos	Web, e-mail, transferência de arquivos, SSH	DNS, streaming ao vivo, VoIP, jogos, DHCP, QUIC

Fonte: adaptado de Kurose e Ross [1] e Forouzan [3].

Questão comentada (estilo ENADE)

Uma conexão TCP opera com cwnd = 20 MSS e ssthresh = 32 MSS quando o transmissor detecta um *timeout*. Considerando o TCP Reno, após o evento os valores de cwnd e ssthresh serão, respectivamente:
(A) 10 MSS e 20 MSS **(B)** 1 MSS e 10 MSS **(C)** 1 MSS e 16 MSS **(D)** 10 MSS e 10 MSS **(E)** 20 MSS e 32 MSS
Resposta: (B). Em qualquer evento de perda, o novo ssthresh é metade da janela **atual** (cwnd/2 = 10), e não metade do ssthresh anterior (erro da alternativa C). Como o evento foi timeout, a cwnd volta a 1 MSS. A alternativa (D) corresponderia à reação a três ACKs duplicados.

7 Camada de Aplicação e Principais Protocolos (HTTP, DNS, SMTP, FTP)

A camada de aplicação é onde estão os programas que as pessoas realmente usam. É importante não confundir a **aplicação** (o navegador Chrome, o Outlook, o FileZilla) com o **protocolo de aplicação** (HTTP, SMTP, FTP). O protocolo é apenas uma parte da aplicação: define os tipos de mensagem, a sintaxe, o significado dos campos e as regras de quando e como responder [1].

7.1 Arquiteturas de aplicação

- **Cliente-servidor**: um servidor sempre ligado, com endereço fixo, atende a pedidos de muitos clientes. Os clientes não conversam entre si. Exemplos: web, e-mail, bancos de dados. Para escalar, usam-se *data centers* e balanceadores de carga.
- **Par a par (P2P)**: os participantes (pares) conversam diretamente, funcionando ao mesmo tempo como clientes e servidores. A grande vantagem é a **autoescalabilidade**: cada novo par traz demanda, mas também capacidade de servir. Exemplo: BitTorrent.

7.2 HTTP: a Web

O **HTTP** (*HyperText Transfer Protocol*) é o protocolo da Web. Uma página é formada por um arquivo HTML base e vários **objetos** referenciados (imagens, scripts, folhas de estilo), cada um identificado por uma **URL**. O HTTP segue o modelo cliente-servidor e é **sem estado** (*stateless*): o servidor não guarda

nenhuma informação sobre pedidos anteriores do cliente. Isso simplifica o servidor e permite atender milhões de clientes.

7.2.1 Formato das mensagens

As mensagens do HTTP/1.1 são texto legível. Uma requisição tem uma linha de requisição, linhas de cabeçalho, uma linha em branco e, opcionalmente, um corpo:

Requisição HTTP

```
GET /graduacao/computacao/index.html HTTP/1.1
Host: www.uscs.edu.br
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: pt-BR
Connection: keep-alive
```

Resposta HTTP

```
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 14:05:12 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 18342
Last-Modified: Mon, 14 Sep 2026 09:30:00 GMT
Cache-Control: max-age=3600

<!DOCTYPE html> <html> ... </html>
```

Tabela 19: Principais métodos HTTP

Método	Função	Seguro?	Idempotente?
GET	Obter um recurso	Sim	Sim
HEAD	Igual ao GET, mas só os cabeçalhos (sem corpo)	Sim	Sim
POST	Enviar dados para processamento (formulário, criação de recurso)	Não	Não
PUT	Criar ou substituir um recurso inteiro	Não	Sim
PATCH	Alterar parte de um recurso	Não	Não (em geral)
DELETE	Remover um recurso	Não	Sim
OPTIONS	Consultar quais métodos são permitidos (usado no CORS)	Sim	Sim

Fonte: adaptado da RFC 9110 [16].

Seguro significa que o método não altera o estado do servidor. **Idempotente** significa que repetir a mesma requisição várias vezes tem o mesmo efeito que fazer uma vez. Esses conceitos são fundamentais no desenho de APIs REST.

Tabela 20: Classes de códigos de status HTTP

Classe	Significado	Exemplos
1xx	Informativo	101 Switching Protocols (usado ao abrir um WebSocket)
2xx	Sucesso	200 OK; 201 Created; 204 No Content
3xx	Redirecionamento	301 Moved Permanently; 302 Found; 304 Not Modified (use seu cache)
4xx	Erro do cliente	400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found; 429 Too Many Requests
5xx	Erro do servidor	500 Internal Server Error; 502 Bad Gateway; 503 Service Unavailable

Fonte: adaptado da RFC 9110 [16].

7.2.2 Conexões persistentes e não persistentes

No HTTP/1.0, cada objeto exigia uma conexão TCP nova (**não persistente**), e cada conexão custa 1 RTT de handshake. O HTTP/1.1 tornou as conexões **persistentes** por padrão: a mesma conexão TCP é reaproveitada para vários objetos.

Estudo de Caso: Tempo para Carregar uma Página

Uma página tem um arquivo HTML base e **10 imagens** pequenas, todas no mesmo servidor. Desconsidere o tempo de transmissão e considere apenas o RTT. Quantos RTTs são necessários em cada caso?
(a) Não persistente, sem paralelismo: cada objeto precisa de 1 RTT para o handshake TCP + 1 RTT para requisição e resposta = 2 RTT. Para 11 objetos: $11 \times 2 = \mathbf{22\ RTT}$.
(b) Persistente, sem paralelismo (sem pipelining): 1 RTT de handshake + 1 RTT para o HTML + 1 RTT para cada imagem: $1 + 1 + 10 = \mathbf{12\ RTT}$.
(c) Persistente com paralelismo: 1 RTT de handshake + 1 RTT para o HTML + 1 RTT para todas as imagens, pedidas de uma vez: **3 RTT**.
Interpretação: com RTT de 50 ms, o tempo cai de 1,1 s para 150 ms. Isso mostra por que a evolução do HTTP se concentrou em reduzir o número de idas e voltas, e não apenas em aumentar a largura de banda.

7.2.3 Evolução: HTTP/2 e HTTP/3

- **HTTP/2** (2015): mensagens em formato **binário**, **multiplexação** de vários fluxos numa única conexão TCP, compressão de cabeçalhos (HPACK) e priorização. Resolve o bloqueio no início da fila (*head-of-line blocking*) na camada HTTP, mas não no TCP: uma única perda de pacote trava todos os fluxos até a retransmissão.
- **HTTP/3** (RFC 9114 [18], 2022): roda sobre o **QUIC** (RFC 9000 [17]), que usa UDP. O QUIC integra a criptografia TLS 1.3 ao transporte, estabelece a conexão em 1 RTT (ou 0 RTT quando o cliente já conhece o servidor), trata cada fluxo de forma independente (uma perda só atrasa o fluxo afetado) e permite **migração de conexão** (o celular troca do Wi-Fi para o 4G sem derrubar a sessão).

7.2.4 Cookies e cache

Como o HTTP não guarda estado, os **cookies** permitem que o servidor "lembre" do usuário: na primeira visita, o servidor envia um cabeçalho `Set-Cookie`; nas próximas, o navegador devolve o valor no cabeçalho `Cookie`. São usados para carrinhos de compra, sessões de login e, de forma polêmica, rastreamento publicitário, o que envolve questões legais tratadas pela LGPD (Lei nº 13.709/2018).

O **cache web** (no navegador ou num servidor *proxy*) guarda cópias de objetos já buscados. Se outro pedido pelo mesmo objeto chega, a resposta sai do cache, reduzindo o tempo de resposta e o tráfego no enlace de acesso. Para verificar se a cópia ainda é válida, usa-se o **GET condicional** (cabeçalhos `If-Modified-Since` ou `If-None-Match`); se nada mudou, o servidor responde `304 Not Modified`, sem reenviar o objeto. É uma aplicação direta do princípio de **localidade de referência** citado na DCN.

7.3 DNS: o catálogo de nomes da Internet

Pessoas preferem nomes (`www.uscs.edu.br`); roteadores trabalham com endereços IP. O **DNS** (*Domain Name System*, RFCs 1034 e 1035 [6]) faz a tradução. Ele é, ao mesmo tempo, um **banco**

de dados distribuído e hierárquico e um protocolo de aplicação que roda principalmente sobre **UDP, porta 53** (usando TCP para respostas grandes e transferências de zona).

Por que não centralizar? Um servidor único seria um ponto único de falha, receberia um volume de consultas impossível de atender, ficaria longe da maioria dos usuários e seria impossível de manter atualizado. A distribuição resolve todos esses problemas.

7.3.1 Hierarquia de servidores

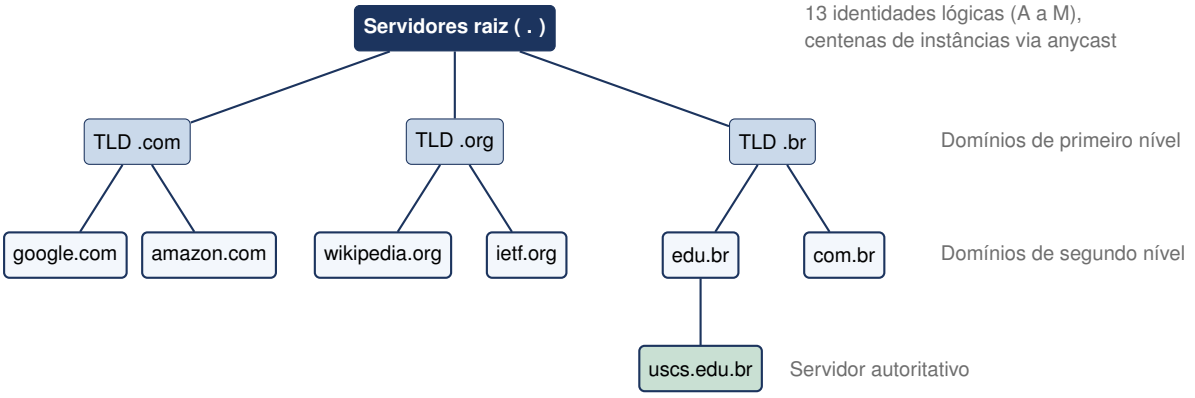


Figura 15: Hierarquia de servidores e espaço de nomes do DNS

Fonte: adaptado de Kurose e Ross [1] e Tanenbaum, Feamster e Wetherall [2].

- **Servidores raiz**: conhecem os servidores de cada TLD.
- **Servidores TLD** (*Top-Level Domain*): responsáveis por `.com`, `.org`, `.br` etc. No Brasil, o `.br` é administrado pelo NIC.br / Registro.br.
- **Servidores autoritativos**: guardam os registros oficiais de uma organização.
- **Servidor DNS local (resolvedor recursivo)**: não faz parte da hierarquia; é o servidor do provedor, da empresa ou público (como 8.8.8.8 ou 1.1.1.1) que faz as consultas em nome do usuário e mantém um cache.

7.3.2 Resolução de um nome

1. O host pergunta ao DNS local: “qual o IP de `www.uscs.edu.br`?”. É uma consulta **recursiva**: o host quer a resposta final.

2–3. O DNS local (sem nada no cache) pergunta à raiz, que responde: “não sei, mas pergunte ao servidor do `.br`, que está em tal IP”.

4–5. O TLD `.br` indica o servidor autoritativo de `uscs.edu.br` (na prática, pode haver ainda um passo intermediário para `edu.br`).

6–7. O autoritativo responde com o registro A (o endereço IP).

8. O DNS local devolve a resposta ao host e guarda tudo em **cache** pelo tempo indicado no **TTL** do registro.

Graças ao cache, a grande maioria das consultas nem chega aos servidores raiz: o endereço dos servidores TLD costuma estar guardado no DNS local.

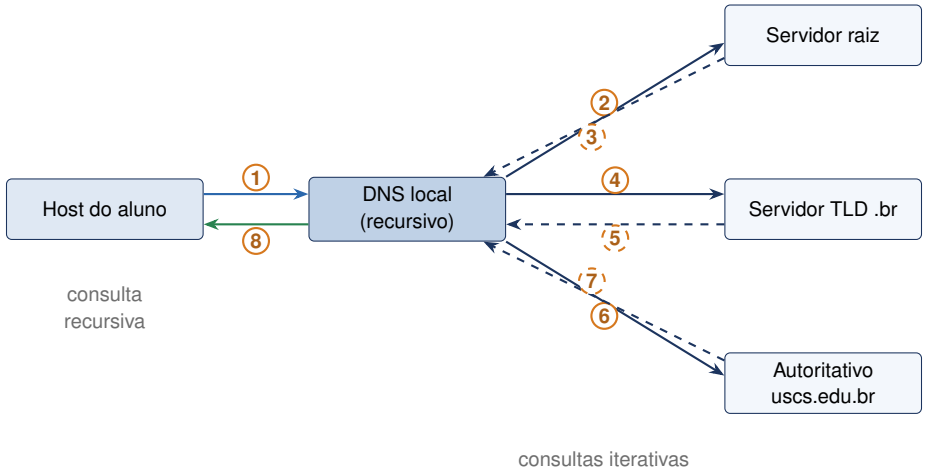


Figura 16: Resolução de `www.uscs.edu.br`: consulta recursiva do host e consultas iterativas do DNS local

Fonte: adaptado de Kurose e Ross [1].

7.3.3 Registros de recursos

O DNS armazena **registros de recursos** (RR) no formato *(nome, valor, tipo, TTL)*:

Tabela 21: Principais tipos de registros DNS

Tipo	Significado	Exemplo de valor
A	Nome → endereço IPv4	200.144.x.x
AAAA	Nome → endereço IPv6	2001:db8::10
CNAME	Apelido → nome canônico	www → servidor1.exemplo.com
MX	Servidor de e-mail do domínio (com prioridade)	10 mail.exemplo.com
NS	Servidor autoritativo do domínio	ns1.exemplo.com
PTR	Resolução reversa (IP → nome)	exemplo.com
TXT	Texto livre (usado em SPF, DKIM, verificação de domínio)	"v=spf1 . . . "
SOA	Informações administrativas da zona	serial, tempos de atualização

Fonte: adaptado da RFC 1035 e de Kurose e Ross [1].

> **Analogia para fixar**
>
> O DNS funciona como a lista telefônica de uma cidade enorme, só que dividida em volumes. Você (o host) pede à telefonista do seu bairro (DNS local) o número de alguém. Ela não tem todos os números, mas sabe qual volume consultar (raiz), que indica a seção certa (TLD), que finalmente mostra o número (autoritativo). Da próxima vez que alguém do bairro pedir o mesmo número, a telefonista já tem anotado num papel ao lado (cache), mas joga o papel fora depois de um tempo (TTL), porque o número pode mudar.

Segurança do DNS. O DNS original não tem autenticação, o que permite ataques de **envenenamento de cache** (respostas falsas que redirecionam usuários para sites maliciosos). O **DNSSEC** acrescenta assinaturas digitais aos registros; o **DoH** (DNS sobre HTTPS) e o **DoT** (DNS sobre TLS) cifram as consultas para proteger a privacidade do usuário.

7.4 Correio eletrônico: SMTP, POP3 e IMAP

O sistema de e-mail tem três componentes principais: os **agentes de usuário** (MUA, como Outlook, Thunderbird ou a interface do Gmail), os **servidores de correio** (MTA), onde ficam as caixas postais, e o protocolo **SMTP** (RFC 5321 [7]), que transfere mensagens entre servidores.

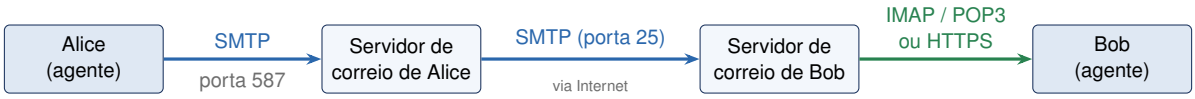


Figura 17: Caminho de um e-mail: SMTP para *enviar* (push) e IMAP/POP3 para *ler* (pull)

Fonte: adaptado de Kurose e Ross [1].

O SMTP é um protocolo de **empurrar** (*push*): quem tem a mensagem a envia. Por isso não serve para Bob buscar seus e-mails, que exige um protocolo de **puxar** (*pull*). O diálogo SMTP é textual, com comandos e respostas numeradas:

Diálogo SMTP simplificado (S = servidor, C = cliente)

```
S: 220 mail.uscs.edu.br ESMTP pronto
C: EHLO cliente.exemplo.com
S: 250 mail.uscs.edu.br
C: MAIL FROM:<alice@exemplo.com>
S: 250 OK
C: RCPT TO:<bob@uscs.edu.br>
S: 250 OK
C: DATA
S: 354 Envie a mensagem; termine com uma linha contendo apenas "."
C: Subject: Reunião do squad
C:
C: Oi Bob, a reunião ficou para quinta às 19h.
C: .
S: 250 Mensagem aceita para entrega
C: QUIT
S: 221 Encerrando
```

Para ler as mensagens, usa-se **POP3** (portas 110/995), que normalmente baixa e apaga as mensagens do servidor, ou **IMAP** (portas 143/993), que mantém tudo no servidor, com pastas e sincronização entre vários dispositivos, por isso é o mais usado hoje.

Hoje, a maioria das pessoas usa *webmail*: o navegador fala HTTPS com o servidor, que por sua vez usa SMTP para enviar para outros domínios. Para combater spam e falsificação de remetente, os domínios publicam no DNS registros **SPF** (quais servidores podem enviar em nome do domínio), **DKIM** (assinatura digital das mensagens) e **DMARC** (política do que fazer quando as verificações falham).

7.5 FTP e transferência de arquivos

O **FTP** (*File Transfer Protocol*, RFC 959 [8]) é um dos protocolos mais antigos da Internet. Sua característica mais marcante é usar **duas conexões TCP**:

- **Conexão de controle (porta 21)**: permanece aberta durante toda a sessão e transporta comandos (`USER`, `PASS`, `LIST`, `RETR`, `STOR`, `QUIT`). Por isso se diz que o FTP envia o controle **fora da banda** (*out-of-band*), ao contrário do HTTP.
- **Conexão de dados**: aberta para **cada** transferência de arquivo ou listagem de diretório.

Diferente do HTTP, o FTP **mantém estado** da sessão: o servidor sabe qual usuário está conectado e qual é o diretório atual.

- **Modo ativo**: o cliente informa uma porta (comando `PORT`) e o **servidor** abre a conexão de dados a partir da sua porta 20. Costuma ser bloqueado por firewalls e NAT do lado do cliente.
- **Modo passivo**: o cliente envia `PASV`, o servidor informa uma porta alta e o **cliente** abre a conexão

de dados. É o modo padrão hoje, justamente por funcionar atrás de NAT.

FTP não é seguro

No FTP clássico, usuário, senha e arquivos trafegam **em texto claro** e podem ser capturados por qualquer um no caminho. As alternativas seguras são o **FTPS** (FTP sobre TLS) e o **SFTP**, que, apesar do nome, é um protocolo completamente diferente: roda sobre o **SSH** (porta 22), usa uma única conexão e é muito comum em rotinas automatizadas de troca de arquivos entre empresas.

7.6 Outros protocolos de aplicação importantes

- **DHCP** (UDP 67/68): distribui configuração IP automaticamente em quatro mensagens, conhecidas pela sigla **DORA**: *Discover* (broadcast do cliente), *Offer* (servidor oferece um IP), *Request* (cliente aceita) e *Acknowledge* (servidor confirma). O endereço é "emprestado" por um tempo (*lease*).
- **SSH** (TCP 22): acesso remoto seguro a terminais, substituto do Telnet, com criptografia e autenticação por senha ou por chave pública.
- **SNMP** (UDP 161/162): monitoramento e gerência de equipamentos de rede.
- **NTP** (UDP 123): sincronização de relógios, essencial para logs, certificados e autenticação.

Questão comentada (estilo ENADE)

Sobre protocolos da camada de aplicação, avalie as afirmações:
I. O HTTP é um protocolo sem estado; por isso, aplicações web usam cookies para manter sessões de usuário.
II. O FTP envia comandos e dados pela mesma conexão TCP, na porta 21.
III. Consultas DNS normalmente usam UDP porque são pequenas e o custo de estabelecer conexão seria proporcionalmente alto.
IV. O SMTP é usado tanto para enviar quanto para ler mensagens da caixa postal do destinatário.
É correto apenas o que se afirma em: **(A)** I e II **(B)** I e III **(C)** II e IV **(D)** I, III e IV **(E)** II, III e IV
Resposta: (B). II está errada porque o FTP usa conexões separadas para controle (21) e dados. IV está errada porque o SMTP é um protocolo de envio (push); a leitura é feita com IMAP, POP3 ou HTTPS.

8 Segurança de Redes: Firewalls, VPNs e Criptografia

A Internet foi projetada originalmente para um grupo pequeno de pesquisadores que confiavam uns nos outros. Segurança não era prioridade: o IP não autentica a origem, o ARP aceita qualquer resposta, o SMTP não verifica o remetente. Hoje, com bancos, governos e hospitais dependendo da rede, a segurança virou um requisito central. A DCN reconhece isso ao exigir que o egresso saiba "identificar e gerenciar os riscos . . . incluindo os aspectos de dependabilidade e segurança" (Art. 5º, §1º, III) e tenha conhecimento das questões "legais e éticas" (Art. 4º, I).

8.1 Propriedades de uma comunicação segura

A base conceitual é a tríade **CID** (ou CIA, em inglês), complementada por outras propriedades [5]:

8.2 Principais ameaças

Tabela 22: Propriedades de segurança e mecanismos que as garantem

Propriedade	Significado	Mecanismo típico
Confidencialidade	Só o destinatário autorizado entende a mensagem	Criptografia (AES, ChaCha20)
Integridade	A mensagem não foi alterada no caminho	Hash, MAC/HMAC, assinatura digital
Disponibilidade	O serviço está acessível quando necessário	Redundância, proteção anti-DDoS
Autenticidade	A origem é realmente quem diz ser	Certificados, senhas, MFA, assinaturas
Não repúdio	O autor não pode negar que enviou	Assinatura digital com chave privada

Fonte: adaptado de Stallings [5] e Kurose e Ross [1].

Tabela 23: Ameaças comuns em redes e propriedade atacada

Ameaça	Descrição	Ataca
Sniffing (escuta)	Captura passiva de pacotes que passam pela rede	Confidencialidade
Spoofing	Falsificação de endereço IP, MAC, DNS ou remetente de e-mail	Autenticidade
Man-in-the-middle	Atacante se coloca entre as partes e lê ou altera o tráfego	Confid. e integridade
DoS / DDoS	Esgotar recursos da vítima (banda, CPU, conexões), muitas vezes com uma *botnet*	Disponibilidade
Malware	Vírus, worms, trojans e *ransomware* (sequestro de dados por criptografia)	Todas
Phishing	Engenharia social para roubar credenciais	Autenticidade
Varredura de portas	Descobrir serviços abertos (ex.: com `nmap`) como etapa de reconhecimento	(preparação)

Fonte: adaptado de Stallings [5] e Tanenbaum, Feamster e Wetherall [2].

8.3 Fundamentos de criptografia

O modelo básico tem uma mensagem em **texto claro** (m), um algoritmo de **cifragem** que, com uma **chave** K, produz o **texto cifrado** $K(m)$, e um algoritmo de decifragem que recupera m. Um princípio fundamental, o **Princípio de Kerckhoffs**, diz que a segurança deve depender apenas do segredo da chave, e não do segredo do algoritmo. Por isso os algoritmos modernos são públicos e analisados exaustivamente pela comunidade.

8.3.1 Criptografia simétrica

Na criptografia **simétrica**, a mesma chave cifra e decifra. É muito rápida, por isso é usada para cifrar o volume de dados das comunicações.

- **Cifras de bloco**: operam em blocos de tamanho fixo. O **AES** (*Advanced Encryption Standard*) usa blocos de 128 bits e chaves de 128, 192 ou 256 bits; é o padrão atual. O **DES** (chave de 56 bits) é considerado inseguro, pois pode ser quebrado por força bruta.
- **Cifras de fluxo**: cifram bit a bit ou byte a byte; o **ChaCha20** é muito usado em celulares, por ser rápido em processadores sem aceleração para AES.
- **Modos de operação**: definem como aplicar a cifra de bloco a mensagens longas. O modo ECB é inseguro (blocos iguais geram cifrados iguais, revelando padrões); modos como **GCM** oferecem confidencialidade e integridade ao mesmo tempo (criptografia autenticada).

O grande problema da criptografia simétrica é a **distribuição de chaves**: como combinar uma chave secreta com alguém que você nunca encontrou, usando um canal que pode estar sendo espionado? Além disso, com n pessoas são necessárias $n(n-1)/2$ chaves diferentes.

8.3.2 Criptografia assimétrica (de chave pública)

Na criptografia **assimétrica**, cada participante tem um **par de chaves**: uma **pública** (K^+), que pode ser divulgada para qualquer um, e uma **privada** (K^-), guardada em segredo. O que uma chave cifra, só a outra decifra:

- Para **confidencialidade**: cifra-se com a chave **pública do destinatário**; só ele, com sua privada, consegue ler.
- Para **assinatura digital**: cifra-se (o hash da mensagem) com a chave **privada do remetente**; qualquer um verifica com a pública dele, e isso prova a autoria.

> **Analogia para fixar**
>
> A chave pública é como um cadeado aberto que você distribui para todo mundo. Qualquer pessoa pode colocar uma mensagem numa caixa e fechar com o seu cadeado, mas só você tem a chave (privada) que abre. Já a assinatura digital é o contrário: é como um carimbo que só você possui; todo mundo conhece o formato do carimbo e consegue conferir se ele é autêntico, mas ninguém consegue falsificá-lo.

Estudo de Caso: RSA com Números Pequenos

O RSA se baseia na dificuldade de fatorar o produto de dois números primos grandes. Para fins didáticos:
Geração das chaves:

1. Escolhem-se dois primos: $p = 5$ e $q = 11$.
2. Calcula-se $n = p \cdot q = 55$ e $z = (p-1)(q-1) = 4 \times 10 = 40$.
3. Escolhe-se e primo em relação a z (sem fatores em comum com 40): $e = 3$.
4. Calcula-se d tal que $e \cdot d \equiv 1 \pmod{z}$: $3 \times 27 = 81 = 2 \times 40 + 1$, logo $d = 27$.
5. Chave pública: $(n, e) = (55, 3)$. Chave privada: $(n, d) = (55, 27)$.

Cifrando a mensagem $m = 8$: $c = m^e \bmod n = 8^3 \bmod 55 = 512 \bmod 55 =$ **17**.
Decifrando: $m = c^d \bmod n = 17^{27} \bmod 55 =$ **8**. ✓
Por que é seguro na vida real? Quem conhece só $(55, 3)$ precisaria fatorar 55 em 5×11 para calcular z e então d. Com 55 isso é trivial, mas com n de 2.048 bits (mais de 600 dígitos decimais), não se conhece algoritmo clássico capaz de fatorar em tempo viável. É um exemplo concreto de como a **complexidade computacional** sustenta a segurança.

8.3.3 Troca de chaves Diffie-Hellman

O algoritmo de **Diffie-Hellman** resolve o problema da distribuição de chaves: permite que duas partes combinem um segredo comum conversando em público. Exemplo didático com o primo $p = 23$ e a base $g = 5$ (ambos públicos):

Um espião que vê p, g, $A = 8$ e $B = 19$ precisaria resolver o **problema do logaritmo discreto** para descobrir a ou b, o que é inviável com números grandes. Importante: o DH puro não autentica ninguém, então é vulnerável a *man-in-the-middle*; por isso, na prática, é combinado com certificados.

Tabela 24: Exemplo numérico de Diffie-Hellman

	Alice	**Bob**
Segredo escolhido	$a = 6$	$b = 15$
Valor enviado em público	$A = 5^6 \bmod 23 = 8$	$B = 5^{15} \bmod 23 = 19$
Cálculo do segredo	$B^a = 19^6 \bmod 23 = \mathbf{2}$	$A^b = 8^{15} \bmod 23 = \mathbf{2}$

Fonte: elaborado pelo SQUAD 1 com base em Stallings [5].

8.3.4 Funções hash e assinatura digital

Uma **função hash criptográfica** $H(m)$ transforma uma mensagem de qualquer tamanho num resumo de tamanho fixo (256 bits no SHA-256). Ela deve ser: **unidirecional** (dado $H(m)$, é inviável achar m), **resistente a colisões** (é inviável achar $m_1 \neq m_2$ com o mesmo hash) e sensível a qualquer alteração (mudar um bit de m muda completamente o resumo, o chamado **efeito avalanche**). MD5 e SHA-1 já tiveram colisões demonstradas e não devem mais ser usados para segurança.

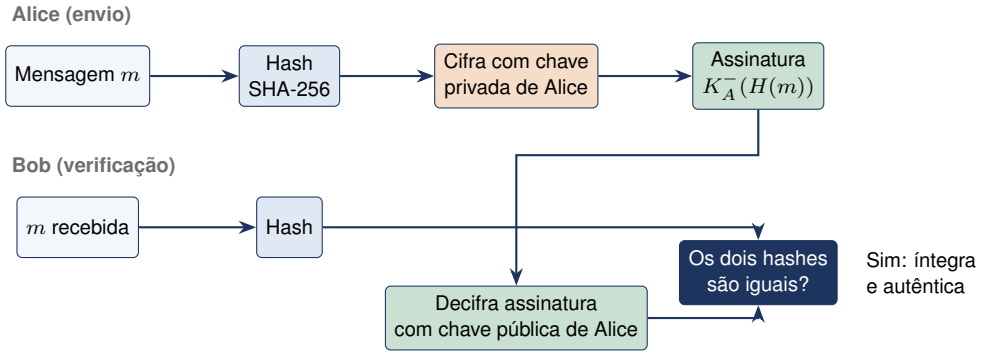


Figura 18: Assinatura digital: integridade, autenticidade e não repúdio

Fonte: adaptado de Stallings [5] e Kurose e Ross [1].

8.3.5 Certificados digitais e PKI

Como Bob sabe que a chave pública que recebeu é mesmo de Alice, e não de um impostor? A resposta é a **Infraestrutura de Chaves Públicas (PKI)**: uma **Autoridade Certificadora (AC)** confiável assina um **certificado digital** (padrão X.509) que vincula uma identidade (um domínio, uma pessoa, uma empresa) a uma chave pública. Navegadores e sistemas operacionais já vêm com as chaves públicas das ACs raiz. No Brasil, a ICP-Brasil é a infraestrutura oficial que dá validade jurídica a assinaturas digitais (e-CPF, e-CNPJ).

8.3.6 TLS: segurança na camada de transporte

O **TLS** (*Transport Layer Security*, sucessor do SSL) é o protocolo que coloca o “S” no HTTPS. Ele combina tudo o que foi visto: no *handshake*, o servidor apresenta seu certificado (autenticidade), as partes fazem uma troca de chaves Diffie-Hellman efêmera (acordo de chave) e, a partir daí, os dados são cifrados com uma cifra simétrica autenticada, como AES-GCM (confidencialidade e integridade). O **TLS 1.3** (RFC 8446 [19]) reduziu o handshake para 1 RTT, removeu algoritmos fracos e tornou obrigatório o **sigilo futuro** (*forward secrecy*): mesmo que a chave privada do servidor vaze no futuro, conversas passadas continuam protegidas.

Na prática, os protocolos usam um esquema **híbrido**: a assimétrica é usada só no início, para autenticar e combinar uma chave de sessão; a simétrica cifra todo o resto.

8.4 Firewalls

Um **firewall** é um sistema (hardware, software ou ambos) que isola a rede interna da externa, permitindo a passagem apenas do tráfego autorizado por uma **política de segurança**. A boa prática é a política de **negação por padrão**: tudo que não é explicitamente permitido é bloqueado.

8.4.1 Tipos de firewall

- **Filtro de pacotes sem estado (*stateless*)**: analisa cada pacote isoladamente, com base em IP de origem/destino, protocolo e portas. É rápido, mas limitado: não sabe se um pacote faz parte de uma conexão legítima. Implementado com listas de controle de acesso (ACLs) nos roteadores.
- **Filtro com estado (*stateful*)**: mantém uma **tabela de conexões**. Um pacote vindo de fora só é aceito se pertencer a uma conexão iniciada de dentro. É o tipo mais comum.
- **Gateway de aplicação (proxy)**: atua na camada 7, entende o protocolo (HTTP, SMTP) e pode filtrar por conteúdo, URL ou usuário.
- **NGFW (*Next-Generation Firewall*)**: combina filtro com estado, inspeção profunda de pacotes (DPI), identificação de aplicações, IPS integrado e integração com identidade do usuário.
- **WAF (*Web Application Firewall*)**: especializado em proteger aplicações web contra ataques como injeção de SQL e XSS.

Tabela 25: Exemplo de regras de um firewall (avaliadas de cima para baixo; vale a primeira que casar)

#	Origem	Destino	Protocolo/Porta	Estado	Ação
1	Qualquer	200.1.1.10 (Web)	TCP 443	–	Permitir
2	Qualquer	200.1.1.20 (Mail)	TCP 25	–	Permitir
3	10.0.0.0/8 (interna)	Qualquer	TCP 80, 443	–	Permitir
4	Qualquer	10.0.0.0/8	TCP	Estabelecida	Permitir
5	10.0.5.0/24 (TI)	200.1.1.0/24	TCP 22	–	Permitir
6	Qualquer	Qualquer	Qualquer	–	**Negar**

Fonte: elaborado pelo SQUAD 1 com base em Kurose e Ross [1].

A ordem das regras importa: se a regra 6 (negar tudo) estivesse em primeiro lugar, nada passaria. A regra 4 é típica de um firewall com estado: permite as respostas das conexões iniciadas pela rede interna.

8.4.2 Arquitetura com DMZ

Servidores que precisam ser acessados pela Internet (web, e-mail, DNS público) não devem ficar na mesma rede dos computadores internos. Eles são colocados numa **DMZ** (*zona desmilitarizada*), uma sub-rede intermediária. Se um servidor da DMZ for comprometido, o atacante ainda precisa passar por outro filtro para chegar à rede interna.

8.4.3 IDS e IPS

Firewalls olham principalmente cabeçalhos. Para detectar ataques escondidos no conteúdo, usa-se o **IDS**, que monitora e **alerta**, ou o **IPS**, que fica no caminho do tráfego e pode **bloquear**. Ambos detectam por **assinatura** (padrões de ataques conhecidos) ou por **anomalia** (desvios do comportamento normal, o que pega ataques novos, mas gera mais falsos positivos). Exemplos: Snort e Suricata.

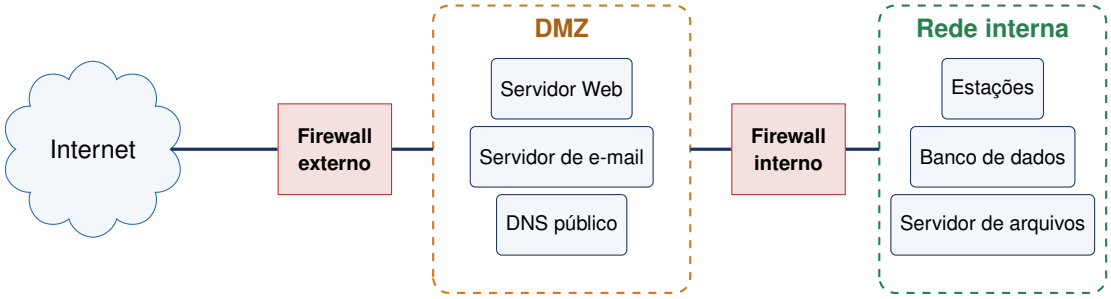


Figura 19: Arquitetura de rede com DMZ entre dois firewalls

Fonte: adaptado de Stallings [5].

8.5 VPNs: redes privadas virtuais

Uma **VPN** (*Virtual Private Network*) cria um **túnel cifrado** sobre uma rede pública (a Internet), fazendo com que dois pontos se comuniquem como se estivessem na mesma rede privada. É "virtual" porque não há um enlace dedicado, e "privada" porque o conteúdo é protegido por criptografia.

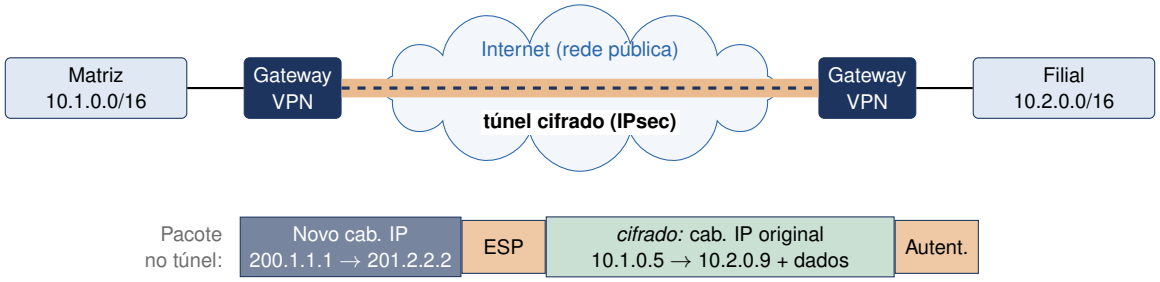


Figura 20: VPN site a site com IPsec em modo túnel: o pacote original inteiro é cifrado e encapsulado num novo pacote IP

Fonte: adaptado de Kurose e Ross [1] e Stallings [5].

8.5.1 Tipos de VPN

- **Site a site**: liga redes inteiras (matriz e filiais) por meio de gateways. Os usuários nem percebem que existe uma VPN.
- **Acesso remoto**: um usuário individual (home office, viagem) se conecta à rede da empresa usando um cliente de VPN no seu computador ou celular.

8.5.2 Protocolos de VPN

IPsec é um conjunto de protocolos que protege a camada de rede. Seus componentes são:

- **AH** (*Authentication Header*): autenticação e integridade, **sem** confidencialidade. É pouco usado, e não funciona bem com NAT.
- **ESP** (*Encapsulating Security Payload*): confidencialidade, integridade e autenticação. É o mais usado.
- **IKE** (*Internet Key Exchange*): negocia algoritmos e chaves, criando as **Associações de Segurança** (SA), que são "contratos" unidirecionais entre as partes.
- **Modo transporte**: protege apenas a carga útil; o cabeçalho IP original é mantido. Usado entre dois hosts.

- **Modo túnel**: protege o datagrama **inteiro**, que é encapsulado num novo datagrama IP. Usado entre gateways (VPN site a site), escondendo até os endereços internos.

Outras opções muito usadas são as **VPNs SSL/TLS** (como o OpenVPN e as VPNs acessadas pelo navegador, que passam facilmente por firewalls por usarem a porta 443) e o **WireGuard**, um protocolo moderno, com código muito menor e criptografia atual, integrado ao núcleo do Linux.

8.6 Defesa em profundidade e Zero Trust

Nenhum mecanismo isolado é suficiente. A estratégia de **defesa em profundidade** empilha várias camadas de proteção: firewall de borda, segmentação por VLANs, IPS, antivírus nos hosts, criptografia, controle de acesso, backups, monitoramento e, principalmente, treinamento dos usuários. Se uma camada falhar, a próxima ainda protege.

O modelo mais recente é o **Zero Trust** (“confiança zero”), descrito pelo NIST na publicação SP 800-207 [21]. Ele abandona a ideia de que tudo dentro do perímetro é confiável (o modelo “castelo e fosso”): cada acesso é verificado individualmente, considerando identidade, estado do dispositivo e contexto, com o princípio do **menor privilégio**. Esse modelo ganhou força com o trabalho remoto e a migração para a nuvem, em que o “perímetro” praticamente deixou de existir.

Questão comentada (estilo ENADE)

Uma empresa deseja que seus funcionários acessem, a partir de casa, um sistema interno que não está exposto à Internet, com garantia de confidencialidade e autenticação. Qual solução atende melhor a esse requisito?
(A) Publicar o sistema com NAT estático e liberar a porta no firewall. **(B)** Implantar uma VPN de acesso remoto com autenticação multifator. **(C)** Usar um IDS na borda da rede. **(D)** Configurar uma VLAN exclusiva para os funcionários remotos. **(E)** Trocar o protocolo de aplicação por FTP.
Resposta: (B). A VPN cria um túnel cifrado (confidencialidade) e a autenticação multifator garante que só funcionários legítimos acessem. A alternativa (A) expõe o sistema na Internet; (C) apenas detecta ataques, não protege o acesso; (D) VLANs funcionam na rede local, não na Internet; (E) o FTP transmite dados em texto claro.

9 Redes Sem Fio, Mobilidade e Tendências (Wi-Fi, 4G/5G, SDN, Nuvem e IoT)

Hoje, o número de dispositivos conectados sem fio supera com folga o de dispositivos cabeados. Celulares, notebooks, relógios, sensores industriais e carros dependem de enlaces de rádio. Ao mesmo tempo, a forma de construir e operar redes mudou: a inteligência está migrando do hardware para o software (SDN), os serviços migraram para a nuvem e bilhões de “coisas” passaram a se comunicar (IoT). Este capítulo apresenta esses temas, que dialogam diretamente com a competência de “adequar-se rapidamente às mudanças tecnológicas” da DCN (Art. 5º, IX).

9.1 Características dos enlaces sem fio

Um enlace sem fio é muito diferente de um cabo [1, 2]:

- **Atenuação do sinal**: a potência cai rapidamente com a distância (no espaço livre, com o quadrado da distância; em ambientes internos, ainda mais rápido) e ao atravessar paredes.
- **Interferência**: outras redes Wi-Fi, Bluetooth, micro-ondas e telefones sem fio disputam as mesmas faixas não licenciadas (2,4 GHz é a mais “congestionada”).

- **Propagação multipercurso**: o sinal reflete em paredes e objetos e chega ao receptor por vários caminhos, com atrasos diferentes, embaralhando os símbolos.
- **Taxa de erros muito maior**: por isso, o Wi-Fi usa confirmação e retransmissão na própria camada de enlace.

A qualidade do enlace é medida pela **relação sinal-ruído** (SNR). Existe um compromisso fundamental: para uma mesma SNR, técnicas de modulação mais agressivas (como 1024-QAM) transmitem mais bits por símbolo, mas têm maior taxa de erro de bit (BER). Por isso os equipamentos fazem **adaptação de taxa**: perto do roteador, usam modulações rápidas; longe, voltam para modulações mais robustas e lentas.

9.1.1 O problema do terminal oculto

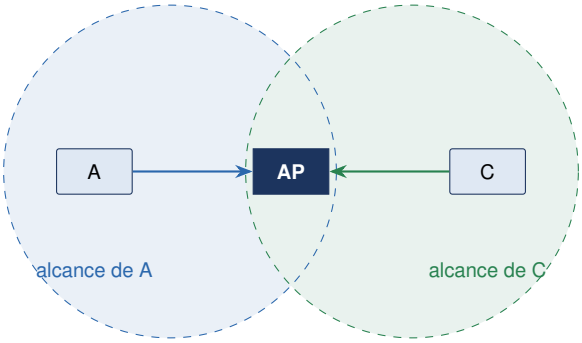


Figura 21: O problema do terminal oculto

Fonte: adaptado de Kurose e Ross [1].

Esse problema e o fato de que um rádio não consegue "ouvir" enquanto transmite (a própria transmissão abafa o sinal recebido) explicam por que o Wi-Fi **não usa detecção de colisão** (CSMA/CD), e sim **prevenção de colisão** (CSMA/CA).

9.2 Redes locais sem fio: IEEE 802.11 (Wi-Fi)

9.2.1 Arquitetura

- **BSS** (*Basic Service Set*): uma célula formada por um **ponto de acesso** (AP) e suas estações. Identificada pelo BSSID (o MAC do AP).
- **ESS** (*Extended Service Set*): vários APs com o mesmo nome de rede (**SSID**) interligados por uma rede cabeada, permitindo que o usuário se mova entre eles (caso típico de uma universidade).
- **Modo ad hoc (IBSS)**: estações se comunicam diretamente, sem AP.

Para entrar numa rede, a estação faz uma **varredura** (passiva, ouvindo os quadros de sinalização *beacon* que os APs transmitem periodicamente, ou ativa, enviando *probe requests*), depois se **autentica** e se **associa** ao AP escolhido.

9.2.2 CSMA/CA

O protocolo de acesso do Wi-Fi funciona assim [1]:

1. Se o canal está livre por um intervalo DIFS, a estação transmite o quadro inteiro.

2. Se o canal está ocupado, a estação sorteia um tempo de recuo aleatório. O contador só diminui enquanto o canal está livre e "congela" quando está ocupado. Quando chega a zero, transmite.
3. O receptor, se recebeu o quadro corretamente, espera um intervalo curto (SIFS) e envia um **ACK**.
4. Se o ACK não chega, a estação conclui que houve colisão ou erro, aumenta a faixa do recuo e tenta de novo.

Opcionalmente, para quadros grandes, a estação pode primeiro enviar um pequeno quadro **RTS** (*Request to Send*); o AP responde com **CTS** (*Clear to Send*), que é ouvido por todas as estações, inclusive as "ocultas", que então ficam em silêncio pelo tempo reservado. Isso resolve o problema do terminal oculto ao custo de um pouco mais de atraso.

9.2.3 Gerações do Wi-Fi

Tabela 26: Evolução dos padrões Wi-Fi

Nome	Padrão	Ano	Faixas	Taxa máx. teórica	Destaques
–	802.11b / g	1999 / 2003	2,4 GHz	11 / 54 Mbps	Popularizaram o Wi-Fi
Wi-Fi 4	802.11n	2009	2,4 e 5 GHz	600 Mbps	MIMO (várias antenas)
Wi-Fi 5	802.11ac	2013	5 GHz	≈ 6,9 Gbps	Canais largos, MU-MIMO no downlink
Wi-Fi 6	802.11ax	2019	2,4 e 5 GHz	≈ 9,6 Gbps	OFDMA, eficiência em ambientes densos
Wi-Fi 6E	802.11ax	2020	+ 6 GHz	≈ 9,6 Gbps	Nova faixa limpa de 6 GHz
Wi-Fi 7	802.11be	2024	2,4, 5 e 6 GHz	≈ 46 Gbps	Canais de 320 MHz, 4096-QAM, operação multienlace

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2] e das especificações da Wi-Fi Alliance.

No Brasil, a Anatel destinou em 2021 toda a faixa de 5.925 a 7.125 MHz (1.200 MHz) para uso não licenciado, permitindo o Wi-Fi 6E e o Wi-Fi 7 com toda a capacidade. Vale lembrar que as taxas da tabela são **máximas teóricas** somando todos os fluxos; um único celular normalmente alcança uma fração delas.

9.2.4 Segurança no Wi-Fi

O **WEP** é inseguro (quebrado em minutos) e o WPA foi só uma transição. O **WPA2** (AES-CCMP) ainda é comum, e o **WPA3** é o recomendado: usa a troca de chaves SAE, resistente a ataques de dicionário *offline*, e oferece sigilo futuro [5].

Em universidades e empresas, o modo **Empresarial** (802.1X) é o mais indicado: cada usuário tem suas próprias credenciais, validadas por um servidor RADIUS. É o caso da rede **eduroam**, usada por instituições de ensino e pesquisa no mundo todo.

9.3 Redes celulares: do 1G ao 5G

As redes celulares dividem a área de cobertura em **células**, cada uma atendida por uma **estação rádio base** (ERB). Células vizinhas usam frequências ou recursos diferentes para evitar interferência, e a mesma frequência é reaproveitada em células distantes. Esse **reuso de frequência** é o que permite atender milhões de usuários com um espectro limitado.

A cada geração, a taxa de pico cresceu cerca de uma ordem de grandeza. Na prática, as taxas percebidas são bem menores, pois dependem da distância à antena, do número de usuários na célula e da faixa usada.

Tabela 27: Gerações das redes celulares

Geração	Tecnologias	Período	Principais características
1G	AMPS	anos 1980	Analógica, só voz, sem segurança
2G	GSM, CDMA	anos 1990	Voz digital, SMS; dados com GPRS/EDGE (dezenas a centenas de kbps)
3G	UMTS, HSPA	anos 2000	Internet móvel de verdade; até dezenas de Mbps com HSPA+
4G	LTE, LTE-Advanced	anos 2010	Rede totalmente IP (voz vira VoLTE); OFDMA; centenas de Mbps
5G	NR (*New Radio*)	anos 2020	Até Gbps, latência de poucos ms, massiva conexão de dispositivos, fatiamento de rede

Fonte: adaptado de Kurose e Ross [1] e Tanenbaum, Feamster e Wetherall [2].

9.3.1 Arquitetura do 4G (LTE)

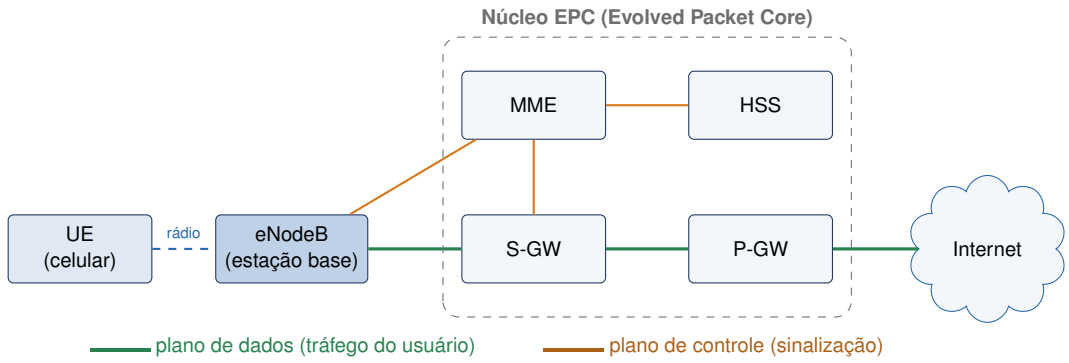


Figura 22: Arquitetura simplificada de uma rede 4G LTE

Fonte: adaptado de Kurose e Ross [1].

- **eNodeB**: a estação base; cuida do rádio e escalona os recursos entre os usuários.
- **MME** (*Mobility Management Entity*): autenticação, controle de mobilidade e estabelecimento dos "túneis" de dados.
- **HSS** (*Home Subscriber Server*): banco de dados com as informações dos assinantes (as chaves que também estão no chip SIM).
- **S-GW e P-GW**: gateways do plano de dados; o P-GW atribui o IP do celular, aplica NAT e é a saída para a Internet.

Um detalhe importante da arquitetura é a separação clara entre **plano de dados** e **plano de controle**, a mesma ideia que aparece nas redes definidas por software.

9.3.2 5G

O 5G foi projetado para atender a três grandes classes de uso definidas pela UIT:

- **eMBB** (*enhanced Mobile Broadband*): banda larga móvel de altíssima velocidade (vídeo 4K/8K, realidade aumentada).
- **URLLC** (*Ultra-Reliable Low-Latency Communications*): latência da ordem de 1 ms e altíssima confiabilidade (cirurgia remota, veículos autônomos, automação industrial).
- **mMTC** (*massive Machine-Type Communications*): até um milhão de dispositivos por km² (sensores e IoT).

Para isso, o 5G usa novas faixas (inclusive ondas milimétricas acima de 24 GHz, com muita capacidade e pouco alcance), **Massive MIMO** com dezenas de antenas e *beamforming* (feixes direcionados a cada usuário), um núcleo (5GC) **baseado em serviços** e rodando em nuvem, e o **fatiamento de rede** (*network slicing*): várias "redes virtuais" com características diferentes sobre a mesma infraestrutura física. No Brasil, o leilão do 5G ocorreu em 2021, e a principal faixa usada é a de 3,5 GHz.

9.4 Mobilidade

Mobilidade significa manter a comunicação enquanto o dispositivo muda de ponto de conexão. O desafio é que o endereço IP identifica a **localização** na rede; se o celular muda de rede, em princípio o IP muda, e as conexões TCP (identificadas pela quádrupla de endereços e portas) caem.

- **Handoff (ou handover)**: transferência da conexão de uma estação base para outra. Nas redes celulares é coordenado pela rede e imperceptível numa ligação. No Wi-Fi, a própria estação decide quando trocar de AP.
- **Roteamento indireto**: os pacotes vão primeiro para a rede de origem (*home*), e um agente lá os redireciona à rede visitada. Simples, mas gera o **roteamento triangular**, ineficiente.
- **Roteamento direto**: o correspondente descobre o endereço atual do móvel e envia diretamente. Mais eficiente, porém mais complexo.

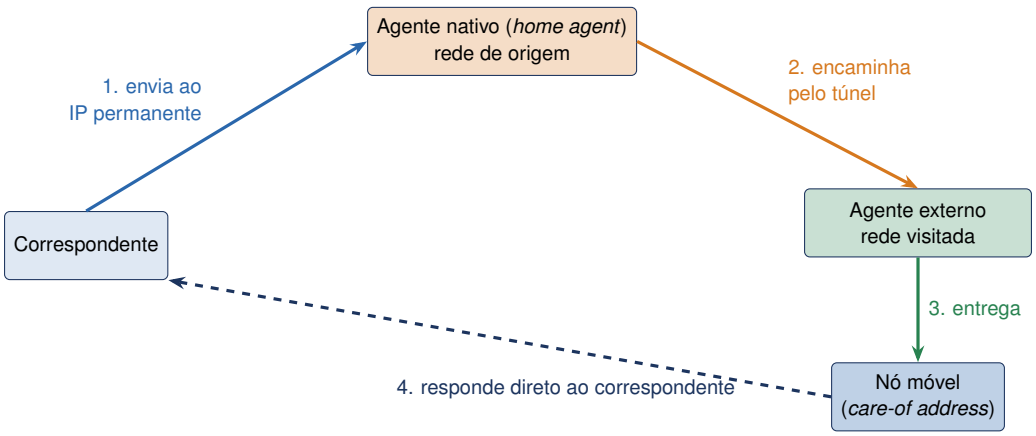


Figura 23: Roteamento indireto no IP Móvel e o "triângulo" formado pelos pacotes

Fonte: adaptado de Kurose e Ross [1].

9.5 Redes Definidas por Software (SDN)

Nas redes tradicionais, cada roteador tem seu próprio plano de controle (roda OSPF, BGP etc.) e seu plano de dados, fortemente acoplados e presos ao software do fabricante. Mudar o comportamento da rede significa configurar equipamento por equipamento, muitas vezes com comandos diferentes para cada marca.

A **SDN** (*Software-Defined Networking*) propõe quatro ideias [1]:

1. **Separação dos planos**: os comutadores ficam só com o plano de dados (encaminhar pacotes rapidamente).
2. **Controle centralizado (logicamente)**: um **controlador** tem a visão global da rede e calcula as regras.
3. **Encaminhamento baseado em fluxos**: as decisões podem considerar vários campos de cabeçalho (MAC, IP, portas, VLAN), não só o IP de destino.

4. **Programabilidade**: aplicações de rede (roteamento, firewall, balanceamento) são programas que rodam sobre o controlador, usando APIs.

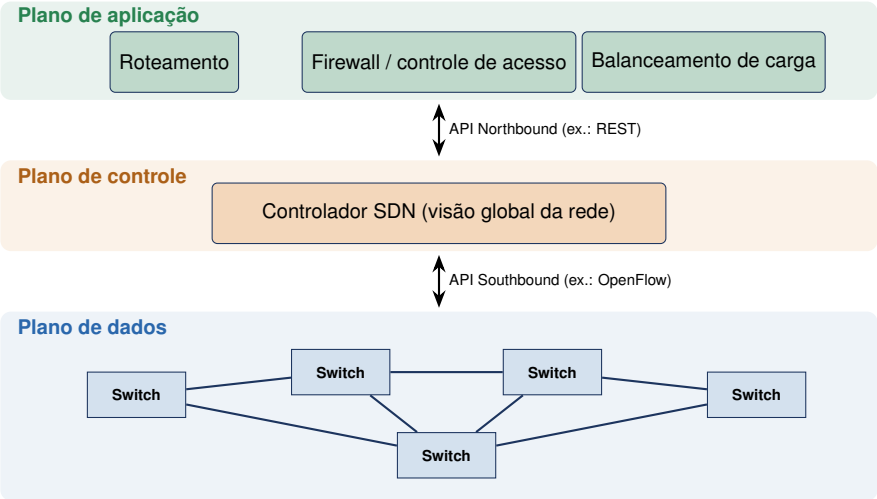


Figura 24: Arquitetura em três planos das Redes Definidas por Software

Fonte: adaptado de Kurose e Ross [1].

9.5.1 OpenFlow e o modelo “casar e agir”

O **OpenFlow** é o protocolo que o controlador usa para instalar regras nos switches. Cada regra da **tabela de fluxos** tem campos para casar (*match*), uma **ação** (*action*), uma prioridade e contadores. Com isso, um mesmo equipamento pode se comportar como switch, roteador, firewall ou NAT, dependendo das regras instaladas.

Tabela 28: Exemplo de tabela de fluxos OpenFlow

Prioridade	Casamento (match)	Ação
300	IP destino = 10.3.0.0/16, TCP porta destino = 22	Descartar (bloqueia SSH para essa rede)
200	IP destino = 10.3.0.0/16	Encaminhar pela porta 4
100	MAC destino = aa:bb:cc:dd:ee:ff	Encaminhar pela porta 2
0	Qualquer (regra padrão)	Enviar ao controlador

Fonte: adaptado de Kurose e Ross [1].

A SDN nasceu na academia (Stanford, por volta de 2008) e hoje é usada em larga escala em data centers e nas redes privadas de grandes empresas, como a WAN B4 do Google. Ela anda junto com a **NFV** (*Network Functions Virtualization*): funções que antes exigiam equipamentos dedicados (firewall, balanceador, núcleo da rede móvel) passam a rodar como software em servidores comuns. O núcleo do 5G é um exemplo direto dessa tendência.

9.6 Computação em nuvem e redes

A **computação em nuvem** é definida pelo NIST (SP 800-145) [20] como um modelo de acesso sob demanda, pela rede, a um conjunto compartilhado de recursos configuráveis, que podem ser rapidamente provisionados e liberados com mínimo esforço de gerência. Suas características essenciais são: autoatendimento sob demanda, amplo acesso pela rede, compartilhamento de recursos, elasticidade e serviço medido (paga-se pelo uso).

- **IaaS** (Infraestrutura como Serviço): o cliente recebe máquinas virtuais, redes e armazenamento

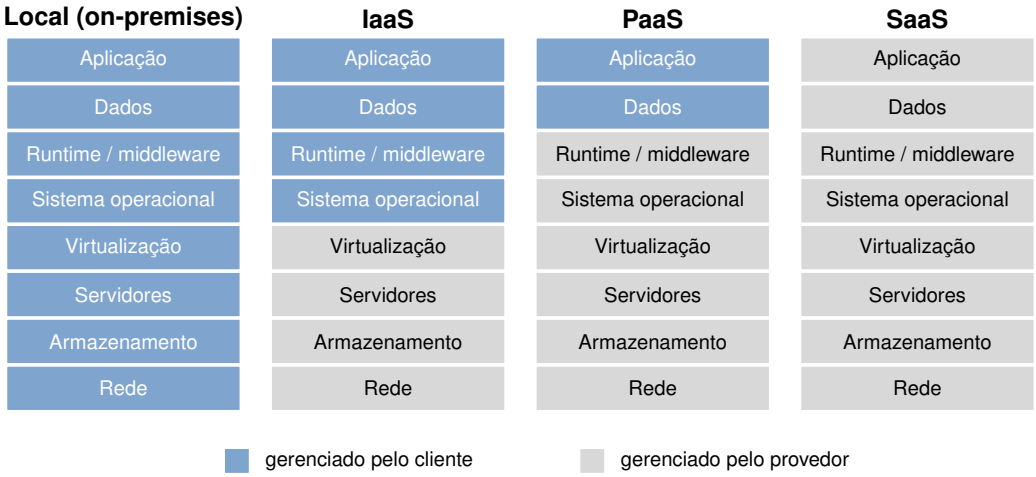


Figura 25: Divisão de responsabilidades nos modelos de serviço em nuvem

Fonte: adaptado da definição do NIST (SP 800-145).

(ex.: Amazon EC2, Azure Virtual Machines).

- **PaaS** (Plataforma como Serviço): o cliente só entrega o código; o provedor cuida do resto (ex.: Google App Engine, Azure App Service).
- **SaaS** (Software como Serviço): o cliente usa a aplicação pronta (ex.: Microsoft 365, Gmail).

Quanto à implantação, a nuvem pode ser **pública**, **privada**, **comunitária** ou **híbrida**. Do ponto de vista de redes, a nuvem trouxe: redes virtuais (VPCs) definidas por software, balanceadores de carga elásticos, interconexões dedicadas entre empresas e provedores e uma enorme demanda por tráfego dentro dos data centers (tráfego “leste-oeste”), que levou a topologias como a **Clos** (ou *leaf-spine*), com muitos caminhos de mesmo custo entre quaisquer dois servidores.

9.6.1 Computação de borda e CDNs

Levar todos os dados até um data center distante aumenta a latência. A **computação de borda** (*edge computing*) processa os dados perto de onde são gerados (numa ERB 5G, numa fábrica, num roteador local), o que é essencial para aplicações URLLC. As **CDNs** (*Content Delivery Networks*), como Akamai e Cloudflare, espalham cópias de conteúdo por milhares de servidores ao redor do mundo e usam DNS ou *anycast* para direcionar cada usuário ao servidor mais próximo. É por isso que um vídeo popular carrega rápido mesmo que o servidor original esteja em outro continente.

9.7 Internet das Coisas (IoT)

A **Internet das Coisas** é a extensão da Internet a objetos do dia a dia e da indústria: sensores, atuadores, eletrodomésticos, medidores de energia, equipamentos agrícolas e médicos. Esses dispositivos normalmente têm **pouca energia** (bateria que deve durar anos), **pouca capacidade de processamento e memória** e enviam **pequenas quantidades de dados**, às vezes de lugares remotos.

Em geral, um sistema de IoT é organizado em quatro camadas: **percepção** (sensores e atuadores), **rede/gateway** (Wi-Fi, 4G/5G, LoRaWAN), **plataforma/nuvem** (armazenamento e gerência) e **aplicação** (painéis, alertas e análise de dados).

Tabela 29: Tecnologias de conectividade para IoT

Tecnologia	Alcance	Taxa	Uso típico
BLE (Bluetooth Low Energy)	Dezenas de metros	Até 2 Mbps	Vestíveis, sensores pessoais
Zigbee / Thread	Dezenas de metros (malha)	250 kbps	Automação residencial
Wi-Fi	Dezenas de metros	Alta	Câmeras, eletrodomésticos com energia
LoRaWAN	Quilômetros	Poucos kbps	Agricultura, cidades inteligentes, monitoramento ambiental
NB-IoT / LTE-M	Quilômetros (rede celular)	Dezenas a centenas de kbps	Medidores, rastreamento
5G mMTC	Quilômetros	Variável	Grande densidade de dispositivos

Fonte: adaptado de Tanenbaum, Feamster e Wetherall [2].

9.7.1 Tecnologias de comunicação

O grupo de tecnologias com longo alcance e baixo consumo (LoRaWAN, NB-IoT, Sigfox) é chamado de **LPWAN** (*Low-Power Wide-Area Network*). Há um compromisso clássico entre três fatores: **alcance**, **taxa de dados** e **consumo de energia**. Não dá para ter os três ao mesmo tempo.

9.7.2 Protocolos de aplicação para IoT: MQTT e CoAP

O HTTP é pesado demais para muitos dispositivos IoT. Dois protocolos se destacam:

- **MQTT**: modelo **publicar/assinar** (*publish/subscribe*) sobre TCP. Os dispositivos não falam entre si diretamente: publicam mensagens em **tópicos** num servidor central chamado **broker**, que as repassa para quem assinou aquele tópico. Tem cabeçalho mínimo de 2 bytes e três níveis de qualidade de serviço (QoS 0: no máximo uma vez; QoS 1: pelo menos uma vez; QoS 2: exatamente uma vez).
- **CoAP**: parecido com um "HTTP enxuto" (métodos GET, POST, PUT, DELETE), mas sobre UDP, pensado para dispositivos muito limitados.

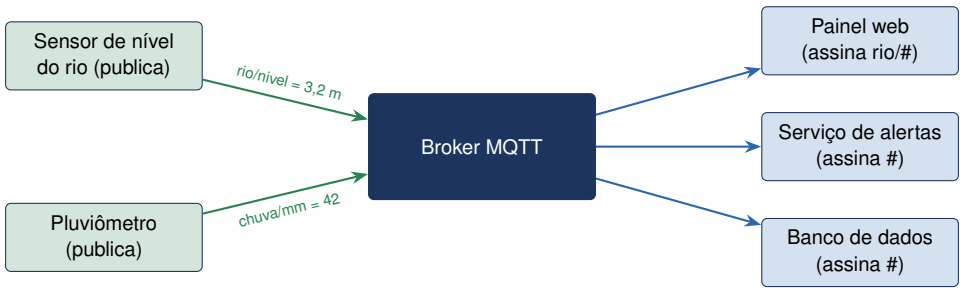


Figura 26: Comunicação publicar/assinar com MQTT num sistema de monitoramento ambiental

Fonte: elaborado pelo SQUAD 1 com base na especificação OASIS MQTT 5.0.

A vantagem do modelo é o **desacoplamento**: o sensor não precisa saber quem vai usar seus dados, e novos consumidores podem ser adicionados sem mexer nos dispositivos.

9.7.3 Desafios da IoT

Os principais desafios são a **segurança** (em 2016, a botnet **Mirai** usou centenas de milhares de câmeras e roteadores com senha padrão para ataques DDoS), a **privacidade** dos dados coletados (LGPD), a **escala** (bilhões de dispositivos reforçam a necessidade do IPv6) e a **energia e interoperabilidade** entre fabricantes.

Questão comentada (estilo ENADE)

Uma prefeitura quer instalar centenas de sensores de nível de rio espalhados por vários quilômetros, alimentados por bateria, que enviam uma leitura de poucos bytes a cada 5 minutos. A combinação de tecnologias mais adequada é:
(A) Wi-Fi 6 e HTTP/2 **(B)** LoRaWAN e MQTT **(C)** Bluetooth Low Energy e FTP **(D)** 5G mmWave e SMTP **(E)** Fibra óptica e HTTP/3
Resposta: (B). O cenário exige longo alcance, baixo consumo e pouquíssimos dados, exatamente o perfil de uma LPWAN como o LoRaWAN; o MQTT é leve e adequado para envio periódico de leituras a um broker. Wi-Fi e BLE têm alcance curto; 5G em ondas milimétricas tem alcance ainda menor e alto consumo; fibra seria cara e inviável para sensores espalhados.

Referências

[1] James F. Kurose e Keith W. Ross. *Redes de Computadores e a Internet: uma abordagem top-down*. 8. ed. São Paulo: Pearson, 2021.

[2] Andrew S. Tanenbaum, Nick Feamster e David J. Wetherall. *Redes de Computadores*. 6. ed. Porto Alegre: Bookman, 2021.

[3] Behrouz A. Forouzan. *Comunicação de Dados e Redes de Computadores*. 4. ed. Porto Alegre: AMGH, 2010.

[4] Douglas E. Comer. *Redes de Computadores e Internet*. 6. ed. Porto Alegre: Bookman, 2016.

[5] William Stallings. *Criptografia e Segurança de Redes: princípios e práticas*. 6. ed. São Paulo: Pearson, 2015.

[6] Paul Mockapetris. *RFC 1034: Domain Names – Concepts and Facilities* e *RFC 1035: Domain Names – Implementation and Specification*. IETF, 1987.

[7] John Klensin. *RFC 5321: Simple Mail Transfer Protocol*. IETF, 2008.

[8] Jon Postel e Joyce Reynolds. *RFC 959: File Transfer Protocol (FTP)*. IETF, 1985.

[9] Conselho Nacional de Educação (CNE). *Resolução CNE/CES nº 5, de 16 de novembro de 2016*. Institui as Diretrizes Curriculares Nacionais para os cursos de graduação na área da Computação. Diário Oficial da União, Brasília, 17 nov. 2016, Seção 1, p. 22-24.

[10] Jon Postel. *RFC 791: Internet Protocol*. IETF, 1981.

[11] Vince Fuller e Tony Li. *RFC 4632: Classless Inter-domain Routing (CIDR)*. IETF, 2006.

[12] Yakov Rekhter et al. *RFC 1918: Address Allocation for Private Internets*. IETF, 1996.

[13] Steve Deering e Robert Hinden. *RFC 8200: Internet Protocol, Version 6 (IPv6) Specification*. IETF, 2017.

[14] Wesley Eddy (ed.). *RFC 9293: Transmission Control Protocol (TCP)*. IETF, 2022.

[15] International Organization for Standardization. *ISO/IEC 7498-1:1994 – Information technology – Open Systems Interconnection – Basic Reference Model*. Genebra: ISO, 1994.

[16] Roy Fielding, Mark Nottingham e Julian Reschke (eds.). *RFC 9110: HTTP Semantics*. IETF, 2022.

[17] Jana Iyengar e Martin Thomson (eds.). *RFC 9000: QUIC – A UDP-Based Multiplexed and Secure Transport*. IETF, 2021.

[18] Mike Bishop (ed.). *RFC 9114: HTTP/3*. IETF, 2022.

[19] Eric Rescorla. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018.

[20] Peter Mell e Timothy Grance. *NIST SP 800-145: The NIST Definition of Cloud Computing*. Gaithersburg: NIST, 2011.

[21] Scott Rose et al. *NIST SP 800-207: Zero Trust Architecture*. Gaithersburg: NIST, 2020.

[22] Vern Paxson et al. *RFC 6298: Computing TCP's Retransmission Timer*. IETF, 2011.

[23] Jon Postel. *RFC 768: User Datagram Protocol*. IETF, 1980.